

09:00-09:50 · 1일자 1차시

LLM Agent & 업무자동화

답을 잘하는 AI에서, 일을 끝내는 Workflow로

Agent와 문제 정의

09:00-09:12 강의 · 09:12-09:20 강사 시연 · 09:20-09:42 개인 실습 · 09:42-09:50 확인

지금 배우는 내용	이 자시에서 이해할 것	함께 챙길 내용
Chatbot과 Agent의 차이 · 자동화 범위	모델보다 먼저 입력·결과·금지 행동을 정한다	자동화할 일 한 문장

자성재 · 무신사 Agentic AI Side PM
서울시립대·아주대 AI 겸임교수

2026.08.23 SUN
09:00-18:00 · LUNCH 14:00-15:00

첫날은 점심시간이 14시입니다



강사의 오후 주요 일정으로 이번 주만 1시간 점심시간을 14:00-15:00에 운영합니다.

11:30-12:00 쉬는 시간에는 빠른 점심 또는 간식을 미리 드시길 권합니다.

12-13시에 점심시간을 드리지 못하게 되었습니다. 불편을 드려 죄송합니다. ㅠ.ㅠ

09:00-12:00	1-3차시	09:00-11:30 수업 (11:30-12:00 쉬는 시간)	빠른 점심·간식 권장
12:00-14:00	4-5차시	12:00-13:40 수업 (13:40-14:00 쉬는 시간)	총 2시간
14:00-15:00	LUNCH	점심시간	이번 주만 14시 시작
15:00-17:30	6-8차시	15:00-17:00 수업 (17:00-17:30 쉬는 시간)	총 2시간 30분
17:30-18:00	Q&A	질문·실습 복구·Exit Ticket	쉬는 시간 뒤 시작

1일차는 여덟 개 차시로 진행합니다

현재 차시의 시간·내용·결과를 확인하고, 쉬는 시간에는 실습 상태를 저장합니다.

시간	슬라이드	진행 내용	강의·시연·실습	수강생이 남길 것
09:00-09:50	1-29	1일자 1차시 · Agent와 문제 정의	강의 12 · 시연 8 · 실습 22 · 확인 8분	자동화할 일 한 문장
09:50-10:40	30-64	1일자 2차시 · Tool Calling과 실행 권한	강의 15 · 시연 10 · 실습 20 · 확인 5분	허용할 도구와 막을 행동
10:40-11:30	65-99	1일자 3차시 · 한국어 데이터와 PBL 사례	강의 14 · 시연 10 · 실습 21 · 확인 5분	입력·결과·근거 예시
11:30-12:00	-	쉬는 시간	1-3차시 종료 후 휴식	바른 점심 또는 간식 권장
12:00-12:50	100-134	1일자 4차시 · Python·VS Code·Git 환경	강의 10 · 시연 15 · 실습 20 · 확인 5분	버전·경로·PR 준비 상태
12:50-13:40	135-168	1일자 5차시 · 안전한 Agent 실행 루프	강의 10 · 시연 10 · 실습 25 · 확인 5분	정상·실패 test 결과
13:40-14:00	-	쉬는 시간	4-5차시 종료 후 휴식	점심 전 실습 상태 저장
14:00-15:00	-	점심시간	1시간	14:55까지 복귀
15:00-15:40	169-202	1일자 6차시 · 무료·로컬 LLM과 Adapter	강의 8 · 시연 10 · 실습 17 · 확인 5분	성공 또는 예상된 실패 결과
15:40-16:20	203-236	1일자 7차시 · LangGraph와 사람 승인	강의 8 · 시연 12 · 실습 15 · 확인 5분	승인·수정·거절 상태
16:20-17:00	237-269	1일자 8차시 · STT와 LangSmith	강의 8 · 시연 10 · 실습 15 · 확인 7분	통질 기준과 READY/HOLD 판단
17:00-17:30	-	쉬는 시간	6-8차시 종료 후 휴식	질문 정리
17:30-18:00	270	Q&A · 실습 복구 · Exit Ticket	질문 15 · 복구 10 · Exit 5분	오늘의 핵심 세 문장

머신러닝·딥러닝·LLM을 거쳐, 지금은 Agent 제품을 만듭니다

차성재 · 무신사 Agentic AI Side PM · 서울시립대/아주대학교 AI 부문 겸임교수

산업	데이터·AI	풀었던 문제	역할
금융	정형 데이터 AutoML·MLOps	은행·카드·보험사의 예측과 의사결정을 자체 AutoML 솔루션으로 자동화	에이젠글로벌 ML Team Leader
의료	대장내시경 영상 DL·CVOps	실시간 영상에서 용종 탐지·진단을 보조하는 의료 AI 서비스를 제작·운영	아이벡스코퍼레이션 AI Engineer
교육	말하기·글쓰기·상담 LLMOps·AICC	7만 명 영어 평가를 3-5일에서 약 10초로 단축하고 상담 전·중·후 AI를 총괄	크레버스 AI Engineer·AI PM
이커머스	고객·상품 맥락 Agentic AI	AID Chat·쇼핑 탐색·Voice·AI 해설·SEO/GEO 등 고객 접점 Agent 제품을 담당	무신사 Agentic AI Side PM

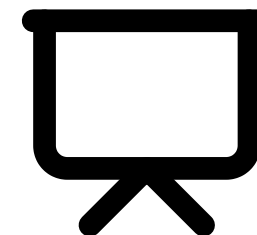
강의·멘토 · 2개 대학원 AI 겸임교수 / KT·Kakao AX / 연구 AX / 재직자·취준생 AI Native / 출제·심사

먼저 네 단어의 역할만 잡고 갑니다

용어	쉽게 말하면	오늘 쓰는 곳
생성형 AI Generative AI	학습한 패턴을 바탕으로 새로운 텍스트·이미지·코드를 만들어내는 AI이다.	회의 원문을 읽고 요약 초안을 만든 뒤 사람이 검토한다.
LLM Large Language Model	아주 많은 텍스트로 학습해 다음에 올 말의 가능성을 계산하는 언어 모델이다.	회의에서 결정·담당자·기한 후보를 구조화한다.
Agent Goal + State + Tools + Loop	목표를 받아 현재 상태를 보고, 도구를 선택하고, 결과를 확인하며 다음 행동을 정하는 실행 구조이다.	파일 읽기 → 요약 → 검증 → 승인 → 기록
Harness Engineering Instructions + Tools + Tests	모델 주변에 지시문·도구·권한·테스트·관측 장치를 설계하는 방식이다.	Codex·Claude에 작은 작업을 주고 테스트와 diff로 결과를 확인한다.

재직자와 구직자는 같은 기술을 다르게 증명한다

재직자는 도입 통제, 구직자는 재현 가능한 포트폴리오를 강조한다.



- 재직자: 보안·업무시간·승인
- 구직자: README·테스트·Git history
- 공통: 실제 demo

제품과 데모가 갈리는 지점 | 데모는 '된다'를 보여주고, 제품은 '안 될 때'를 설명한다.

수업은 네 번의 움직임을 반복합니다

먼저 보고, 그대로 따라하고, 일부러 틀려본 뒤, 성공 증거를 남깁니다.

1

강사 화면을 먼저 봅니다

완성 결과와 클릭·명령 순서를 먼저 확인합니다.

2

같은 화면을 따라 만듭니다

한 단계씩 멈추며 내 화면과 비교합니다.

3

일부러 한 번 틀려봅니다

입력값 하나를 바꿔 실패 메시지를 직접 확인합니다.

4

다시 실행해 확인합니다

원상 복구한 뒤 test와 결과 파일을 남깁니다.

막히면 “안 돼요”보다 네 가지를 알려주세요

“안 돼요”

어디서부터 확인해야 할지
알기 어렵습니다

대신 아래 네 가지를 알려주세요

- | | | |
|----|---------|-----------------|
| 01 | 첫 오류 문장 | 화면에 보이는 그대로 |
| 02 | 직전 행동 | 누른 버튼 또는 입력한 명령 |
| 03 | 내 실행 환경 | 운영체제와 Python 버전 |
| 04 | 기대한 결과 | 원래 나와야 했던 화면 |

예시 | macOS · Python 3.11 · pip install 뒤 ModuleNotFoundError · notebook가 열릴 것으로 기대

실습 속도가 달라도, 다음 행동은 분명합니다

빠르거나 느린 것이 아니라, 지금 내 상태에 맞는 행동 하나를 고르면 됩니다.

지금 내 상태	지금 할 일	다음에 남길 것
바로 다시 실행할 수 있어요	혼자 한 번 더 실행하고 결과를 비교합니다.	성공 화면 1장
장표를 따라가는 중이에요	현재 표시된 단계까지만 끝내고 강사 화면과 비교합니다.	현재 단계 체크
오류에서 멈췄어요	채팅창에 첫 오류 한 줄을 남긴 뒤 복구 파일에서 다시 시작합니다.	오류 문장 1줄
먼저 끝냈어요	실패 입력력을 하나 더 넣고 왜 막혔는지 기록합니다.	반례와 이유

막혔을 때 조용히 기다리지 않아도 됩니다. 첫 오류 한 줄이 가장 좋은 질문입니다.

실습 완료는 “작동했다”가 아니라 “다시 확인할 수 있다”입니다

다음 네 가지 중 하나라도 없으면, 아직 실습이 끝난 것이 아닙니다.

완료 체크

- ✓ 결과 파일이 저장되어 있습니다
- ✓ 같은 명령을 다시 실행해도 성공합니다
- ✓ test 또는 schema 검사가 통과합니다
- ✓ 왜 안전한지 한 문장으로 설명할 수 있습니다

다시 확인할 수 있는 화면

```
$ python3 -m src.day1_agent  
status: SUCCESS
```

```
$ python3 -m pytest -q  
10 passed
```

```
$ git status --short  
M src/day1_agent.py
```

오늘 남길 것 | 실행 화면 1장 + test 결과 1줄 + 내가 설명한 안전 기준 1문장

강사의 완성 흐름은 이 네 화면에서 시작합니다

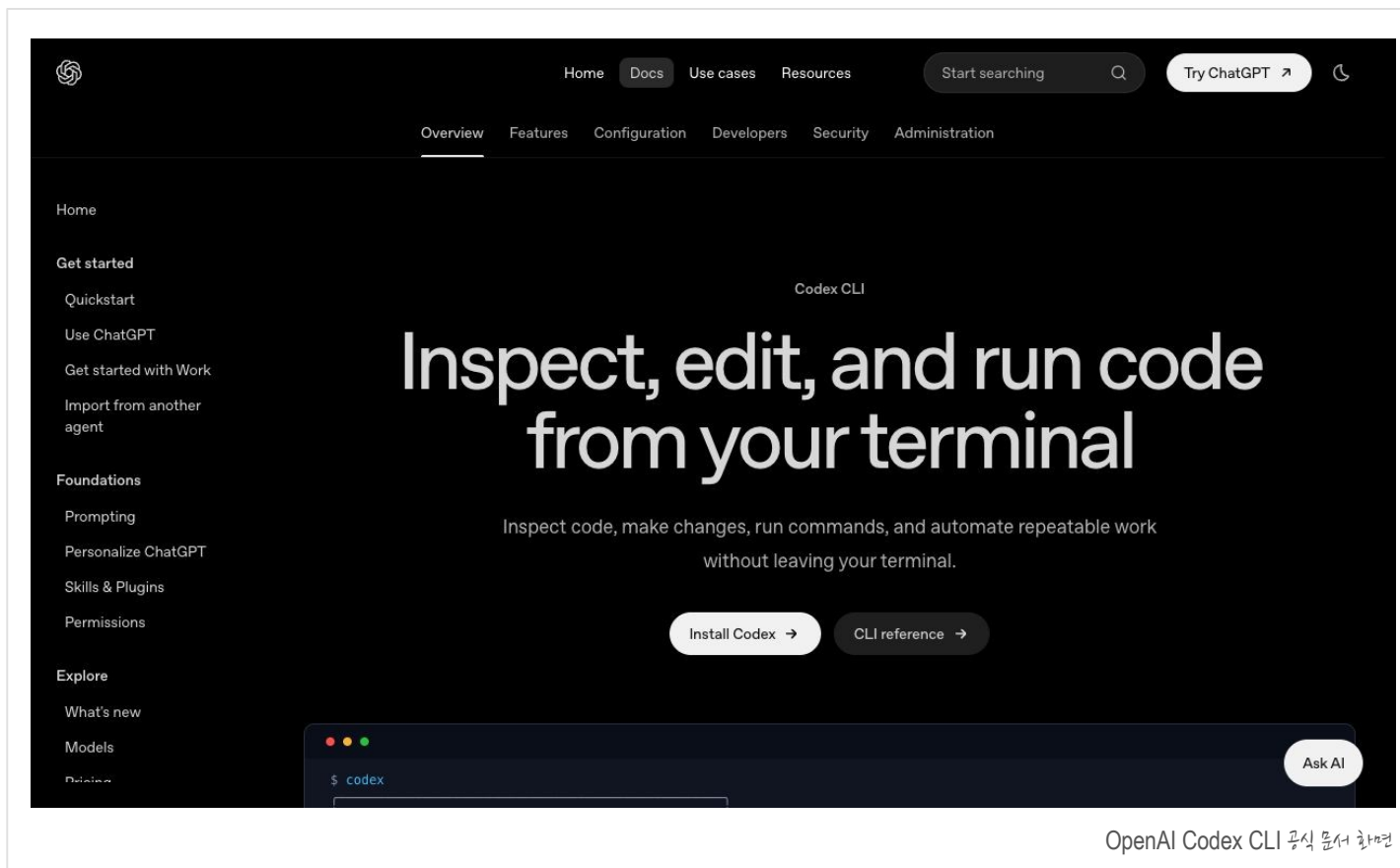
Agent 도구 → 저장소 → 실제 코드 → test 증거로 이어지는 출발 경로

도구·서비스	도메인·역할	이번 강의에서 쓰는 이유	유사 도구와의 차이
VS Code	IDE·주 실행 공간	코드·터미널·Notebook·Git을 한 화면에서 연결	PyCharm보다 가볍고 확장 기반
Jupyter	설명과 셀 실행	처음 배우는 사람이 단계별 결과를 바로 확인	Colab은 설치가 쉽지만 세션·자원 제약
Git·GitHub	버전·포트폴리오	diff·commit·README로 완료 증거를 남김	Drive 공유보다 변경 이력·이전 선택
Codex·Claude	Coding Agent·가속	spec→diff→test 데모와 코드 검토를 빠르게 반복	필수 도구가 아닌 선택적 Harness

Codex CLI · 저장소를 읽고 실행하는 Agent

이 화면에서 꼭 볼 것

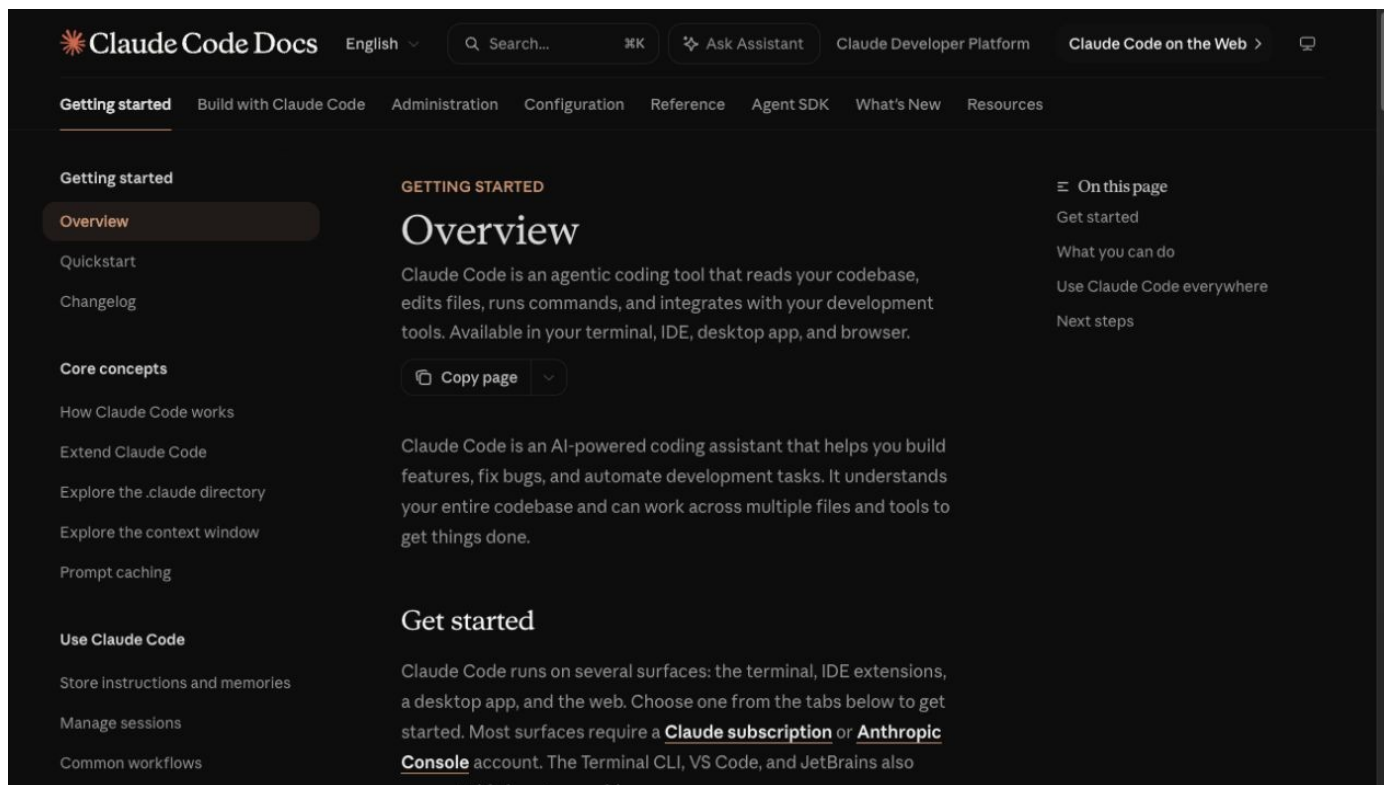
- 작업 디렉터리에서 시작
- 요구사항을 파일·테스트로 구체화
- diff와 명령 결과를 사람이 검토
- 학생 필수 경로는 로컬 코드



Claude Code · 터미널 중심 Harness

이 화면에서 꼭 볼 것

- 코드베이스 맥락 읽기
- 작업을 작은 단계로 지시
- 변경·테스트·설명 확인
- 구독·정책은 별도 확인

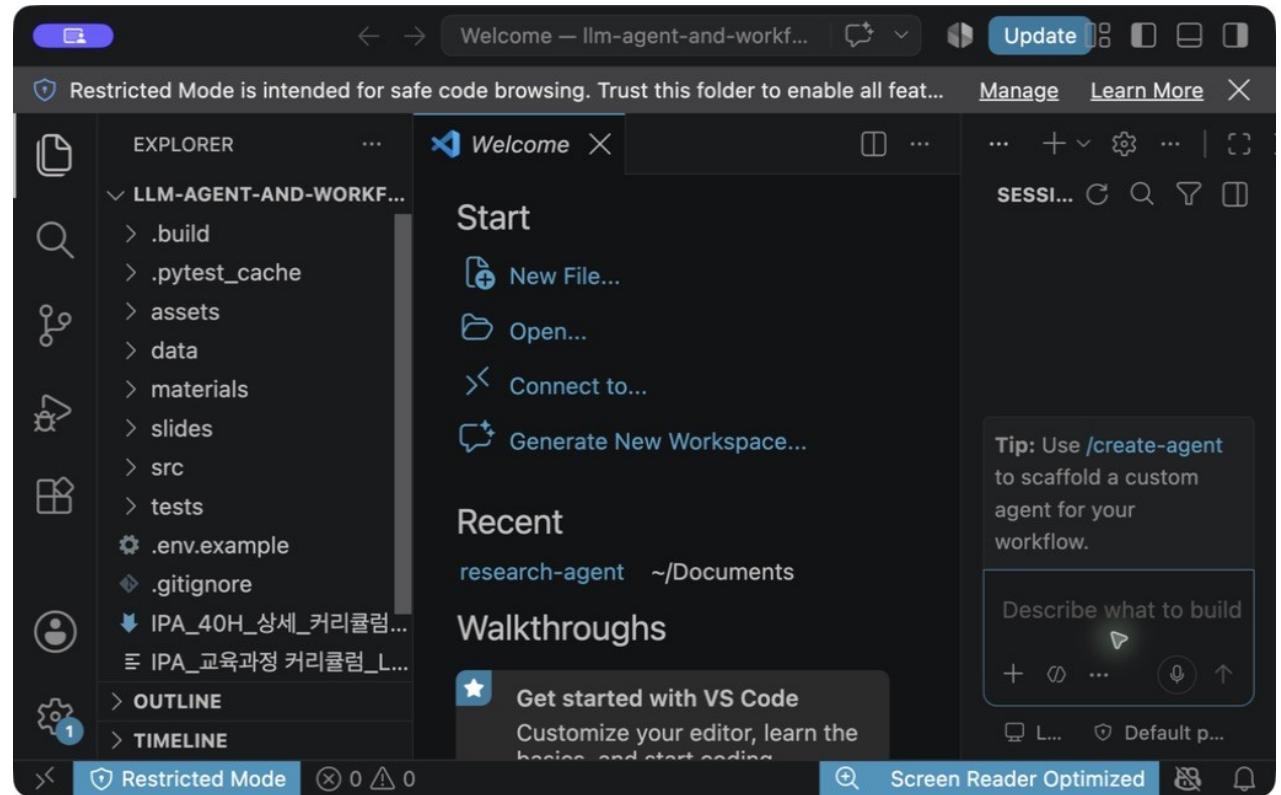


Anthropic Claude Code 공식 Overview

실제 VS Code 저장소 · 수업의 출발 화면

이 화면에서 꼭 볼 것

- src·tests·materials를 먼저 찾기
- .env는 열지 않기
- 강사와 같은 폴더 구조 확인
- 실습 시작 화면으로 사용



로컬에서 만든 강의용 저장소

실제 Agent 코드 · 표준 라이브러리부터

이 화면에서 꼭 볼 것

- 처음에는 framework 없이 제어 루프
- 읽기 도구만 허용
- 오류를 결과 객체로 정규화
- Ollama는 선택 adapter

```

src > day1_agent.py
1  """Day 1: a safe, dependency-free tool-calling loop
2
3  The module intentionally uses only the Python standard
4  learner run the Agent control loop before LangChain
5  introduced. A real Ollama adapter is included as an
6  """
7
8  from __future__ import annotations
9
10 from dataclasses import asdict, dataclass
11 from datetime import date
12 import hashlib
13 import json
14 import os
15 from pathlib import Path
16 import re
17 from typing import Any, Callable
18 from urllib import error, request
19

```

VS Code에서 src/day1_agent.py

작업 요청을 먼저 spec으로 고정

코드 전체보다 경계 한 줄을 먼저 봅니다

- 목표를 한 문장으로
- 입력·출력 형식
- 허용 행동
- 중단과 완료 증거

```
goal: 안전한 파일 읽기 Agent
input: 공개 txt/md
allowed_tools: [read_public_text]
stop: validation_error | success
evidence: pytest + output json
```

Agent의 최소 반복 구조

코드 전체보다 경계 한 줄을 먼저 봅니다

- 판단과 실행을 분리
- 매 step 결과 관찰
- 무한 반복 금지
- state로 감사 가능

```
while state.status == "RUNNING":  
    call = planner(state)  
    checked = validate(call)  
    result = execute(checkered)  
    state = observe(state, result)  
return state
```

Day 1 완료조건을 명령어로 표현

코드 전체보다 경계 한 줄을 먼저 봅니다

- 실행 결과
- 테스트
- 변경 범위
- 설명 가능한 diff

```
python3 -m src.day1_agent
python3 -m pytest -q
git status --short
git diff -- src tests

# success: 10 passed
```

저장소가 말해주는 역할 분리

코드 전체보다 경계 한 줄을 먼저 봅니다

- 코드와 데이터 분리
- 노트북과 모듈 분리
- 테스트를 1급 산출물로
- 장표와 파일명 연결

```
src/          # 실행 코드
tests/        # 실패·정상 증거
materials/    # 따라하기 notebook
data/         # 공개·합성 입력
slides/       # 수업 화면
```

강사 회의음성 Demo · 실패해도 수업은 계속

코드 전체보다 경계 한 줄을 먼저 봅니다

- 오디오가 있으면 Local STT
- 실패하면 같은 60초 transcript
- 사전 생성 JSON 즉시 확인
- 자동 메일 false · 사람 승인 true

```
python3 -m src.meeting_demo \  
  --audio data/demo_meeting.wav \  
  --transcript data/demo_meeting_transcript.txt \  
  --out output/day1-demo  
  
# outputs  
# transcript.json + meeting_result.json
```

실습 1 · 강사 Demo를 보고 내 업무를 고릅니다

06 MIN

강사 화면을 보며
각자 진행합니다

막히면 채팅창에
첫 오류 한 줄만
남겨주세요

- 1 강사가 준비한 짧은 회의 음성을 코드에 넣습니다.
- 2 전사문·회의 요약·할 일 JSON이 만들어지는 화면을 봅니다.
- 3 자동 메일 발송이 차단되는 결과를 확인합니다.
- 4 같은 형식으로 내가 자동화할 일을 한 문장으로 적습니다.

예시 | 강사 시연 | demo_meeting.wav → transcript.json → meeting_result.json / 자동 메일 발송은 하지 않음

완료 증거 | 입력 1개 + 결과 1개 + 금지 행동 1개

실습 2 · 오늘 사용할 예시를 혼자 정합니다

05 MIN

강사 화면을 보며
각자 진행합니다

막히면 채팅창에
첫 오류 한 줄만
남겨주세요

- 1 재직자는 회사명·고객 정보를 5번 반복 업무를 고릅니다.
- 2 구직자는 제공된 공개·합성 예시 중 하나를 고릅니다.
- 3 입력 자료와 원하는 결과를 한 줄씩 적습니다.
- 4 이후 실습에서 같은 예시를 계속 사용합니다.

예시 | 회의 음성 → 결정·담당자·기한이 있는 회의 기록

완료 증거 | 내가 사용할 예시 한 문장

실습 3 · 최종 데모 역산

08 MIN

강사 화면을 보며
각자 진행합니다

막히면 채팅창에
첫 오류 한 줄만
남겨주세요

- 1 최종 입력 한 건을 정한다.
- 2 보여줄 출력 JSON을 그린다.
- 3 사람이 승인할 지점을 표시한다.
- 4 LangSmith에서 보고 싶은 실패를 적는다.

완료 증거 | 입력 → 출력 → 승인 sketch

실습 4 · 환경 신호등

06 MIN

강사 화면을 보며
각자 진행합니다

막히면 채팅창에
첫 오류 한 줄만
남겨주세요

- 1 Python·VS Code·Git 설치 여부를 표시한다.
- 2 관리자 권한 여부를 표시한다.
- 3 RAM·디스크 제약을 적는다.
- 4 GREEN/YELLOW/RED lane을 선택한다.

완료 증거 | 개인 환경 진단표

실패 1 · 문제 범위가 너무 크다

현업에서 자주 깨지는 방식

- 전자·요약·메일·ERP를 한 번에
- 사용자가 여러 명
- 정답 기준 없음
- 데모 입력도 미정

제가 먼저 고치는 순서

- 한 사용자·한 입력
- 한 구조화 출력
- 한 승인 행동
- 한 개의 golden example

실패 2 · 실제 회사 데이터를 가져온다

현업에서 자주 깨지는 방식

- 고객명·이메일 포함
- 비공개 회의 업로드
- 회사 repo token 사용
- 교육 계정에 trace 전송

제가 먼저 고치는 순서

- 공개·합성 데이터
- 이름·금액 범주화
- sandbox repo
- 민감 필드 masking

실패 3 · 모델 선택부터 시작한다

현업에서 자주 깨지는 방식

- 벤치마크 순위만 비교
- 큰 모델 다운로드 대기
- 문제 정의 지연
- 실패 조건 미정

제가 먼저 고치는 순서

- fixture로 흐름 먼저
- 작은 모델은 선택
- 계약과 test 우선
- provider 교체 가능

재직자 · 현업 PoC 승인 관점

현업 질문

- 어떤 시간을 줄이나?
- 어디서 사람이 승인하나?
- 어떤 데이터가 나가나?
- 장애 시 누가 복구하나?

남길 증거

- before/after 시간 가설
- approval policy
- data boundary
- 운영 runbook

Gate 1 · 범위를 줄일 수 있습니까?

상황: “회의 업무를 전부 자동화하자”는 요청을 받았습니다. 오늘 만들 수 있는 한 조각으로 잘라봅니다.

1

사용자 한 명과 입력 한 건을 무엇으로 정할까?

2

AI가 하면 안 되는 행동을 어디까지 막을까?

3

완료를 증명할 파일·test·demo는 무엇일까?

Tool Calling과 실행 권한

09:50-10:05 강의 · 10:05-10:15 코드 시연 · 10:15-10:35 개인 실습 · 10:35-10:40 확인

지금 배우는 내용	이 차시에서 이해할 것	함께 챙길 내용
Tool schema · 허용 목록 · 실행 전 검사	모델의 제안과 실제 실행 권한은 다르다	허용할 도구와 막을 행동

모델의 JSON은 아직 실행이 아닙니다

모델의 제안을 실제 행동으로 바꾸기 전에 누가 마지막으로 검사하는가?

요청

tool name과 arguments를 분리

계약

required·type·추가 필드를 검사

권한

allowlist와 workspace 경계를 확인

실행

registry에 등록된 함수만 호출

기록

result와 audit event를 함께 남김

실행 전에 알아야 할 네 가지 계약

용어	쉽게 말하면	오늘 쓰는 곳
Tool Calling Structured Tool Request	모델이 자유문장 대신 도구 이름과 인자를 구조화해 요청하는 방식이다.	read_public_text(path=...) 요청을 만들고 실행기는 따로 검증한다.
Schema Data Contract	입력과 출력에 어떤 필드가 필요하고 어떤 타입인지 정한 계약이다.	path는 문자열, action_items는 배열, due는 날짜 또는 null
Validation Contract Check	실행 전에 입력이 Schema와 정칙식을 지키는지 검사하는 단계이다.	허용 확장자가 txt/md가 아니면 validation_error로 중단
Side Effect External Change	파일 쓰기·메일 발송·DB 변경처럼 외부 상태를 바꾸는 행동이다.	회의 요약을 Slack에 게시하는 행동은 승인 뒤 한 번만 실행한다.

Tool Calling은 실행 권한의 계약

모델의 JSON은 제안일 뿐, 실행 허가는 애플리케이션이 결정한다.



- name·description
- arguments schema
- allowlist·policy

권한 설계의 핵심 | Tool schema가 친절할수록 모델은 똑똑해 보이고, allowlist가 단단할수록 제품은 안전해진다.

Schema는 문서이자 방화벽

필수 필드·타입·열거값·추가 필드 금지를 코드로 고정한다.

- required
- type·enum
- additional properties

Registry가 실행 가능한 세계를 제한한다

모델이 아는 도구와 실제로 등록된 도구를 분리한다.

- TOOL_SCHEMAS
- TOOL_REGISTRY
- unknown tool 차단

최소 권한은 기능을 줄이는 설계

범용 shell보다 목적별 좁은 도구가 더 안전하고 평가하기 쉽다.

- read_public_text
- 확장자 제한
- workspace 경계

권한 설계의 핵심 | 오류 메시지도 제품 계약이다. 복구할 수 없는 오류는 관측되지 않은 오류다.

Validation은 실행 전에 끝낸다

누락·타입·예상 밖 인자는 side effect 전에 거부한다.

- missing
- unexpected
- wrong type

오류도 정상적인 결과 형식이다

traceback 대신 `ok·error_code·message`를 downstream에 전달한다.

- VALIDATION_ERROR
- NOT_FOUND
- POLICY_BLOCKED
- TOOL_RUNTIME_ERROR

Read와 Write는 위험도가 다르다

외부 쓰기는 승인·중복 방지·감사 로그가 추가로 필요하다.

- read: 자동 가능
- draft: 검토
- publish: 승인 필수

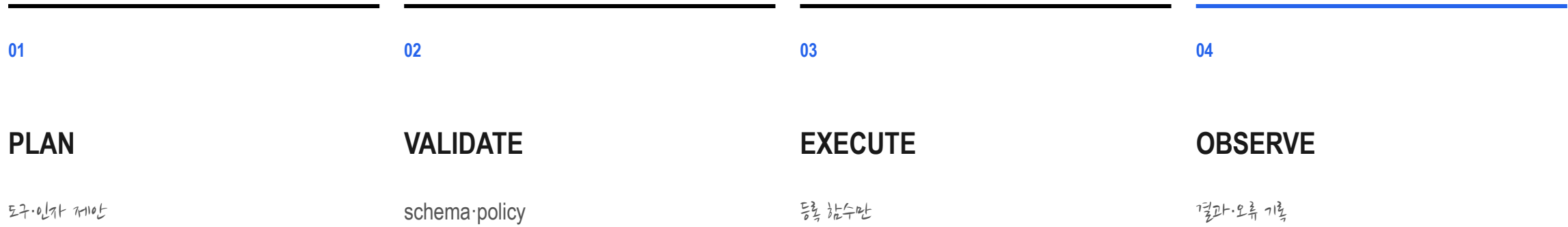
Audit event가 설명 가능성을 만든다

누가 무엇을 요청했고 무엇이 차단됐는지 구조화해 남긴다.

- input hash
- tool·args
- decision·result
- timestamp

권한 설계의 핵심 | 외부 쓰기 권한은 기능이 아니라 책임의 이전이다.

Tool 호출의 4단계



Tool Prompt와 Schema를 만드는 순서

01	02	03	04
SYSTEM	TASK	TOOLS	EXAMPLE
역할·허용·금지 행동	입력과 원하는 결과	name·설명·인자 schema	정상 호출·차단 호출

파일 도구의 경로 검증

01

JOIN

workspace+path

02

RESOLVE

절대경로 정규화

03

CHECK

root 하위인지

04

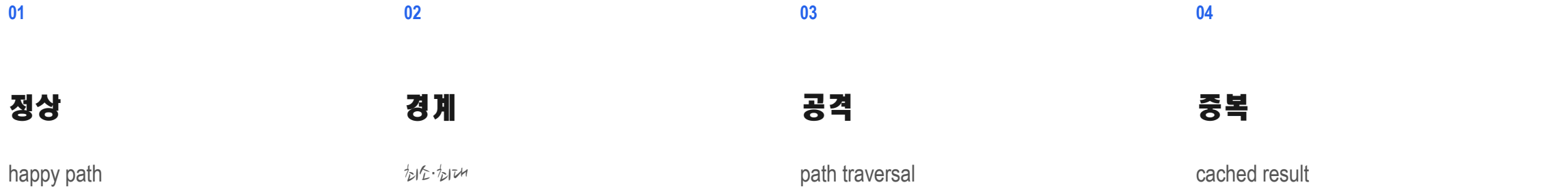
ALLOW

확장자 존재

오류 분류 결정

01	02	03	04
정책	사용자	일시	미지
재시도 없음	입력 수정	제한 retry	중단·trace

테스트 설계 순서



‘실행 허가’가 생기는 화면을 순서대로 봅니다

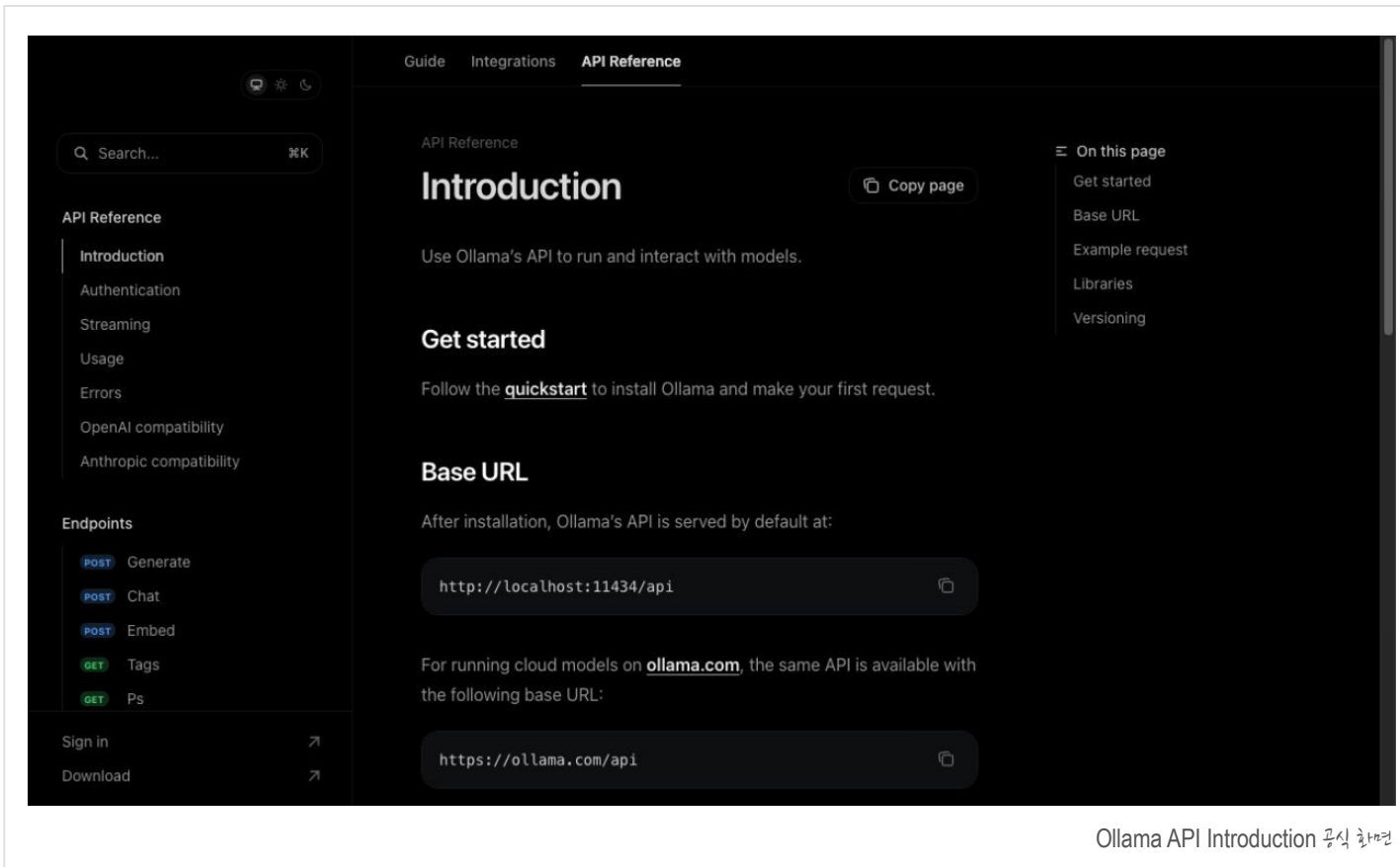
API 형식 → 인증 경계 → test 규칙 → 실제 통과 화면

01	요청·응답 형식 확인	base URL·payload
02	인증을 코드 밖으로 분리	환경 변수·최소 권한
03	실패를 test 함수로 고정	assert·error code
04	정상·예외를 한 번에 실행	10 passed

Ollama API · 호출 형식을 공식 문서로 확인

이 화면에서 꼭 볼 것

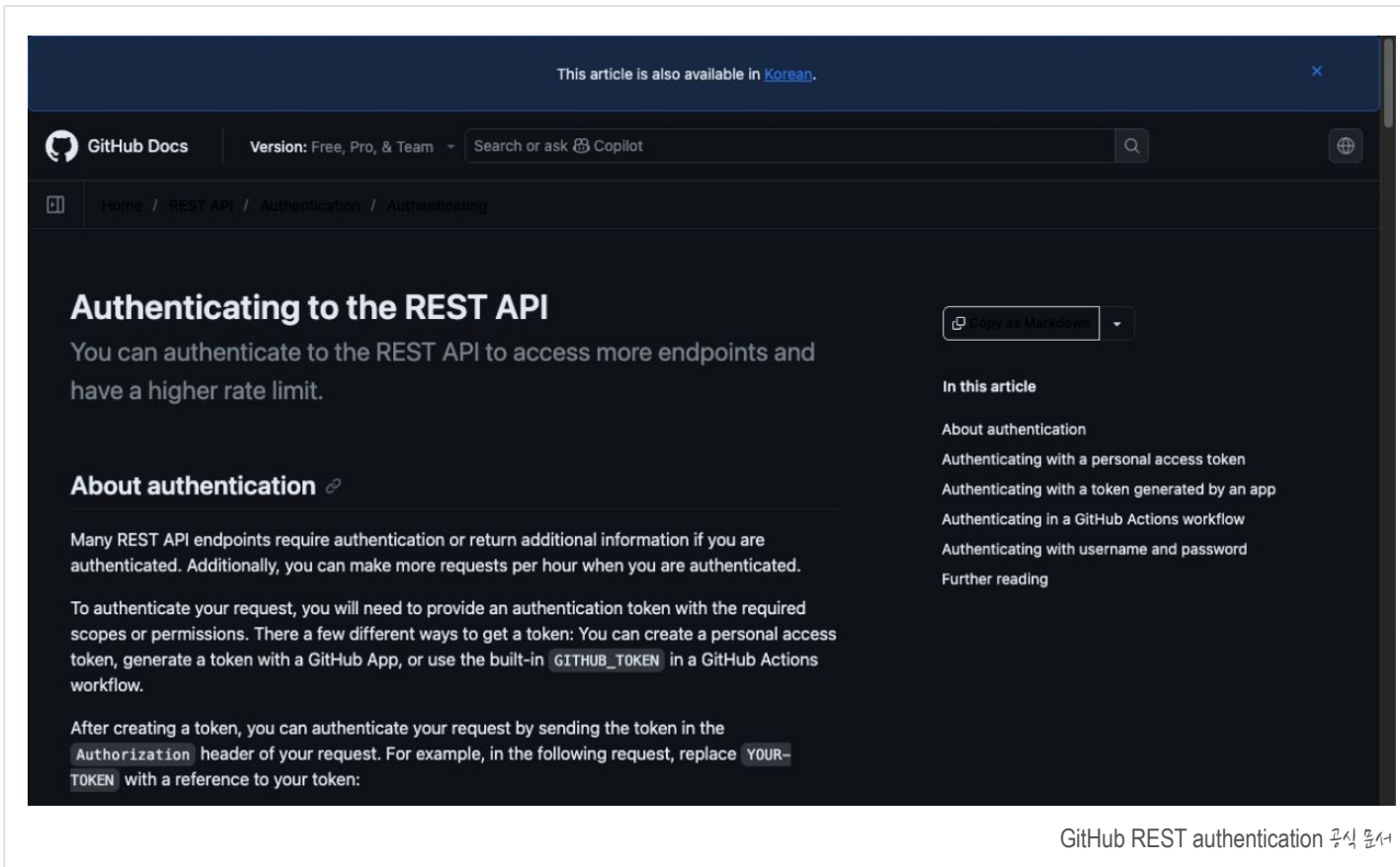
- base URL 확인
- request/response 형식
- timeout·stream 선택
- adapter 뒤에 감추기



GitHub REST · 인증은 코드와 분리

이 화면에서 꼭 볼 것

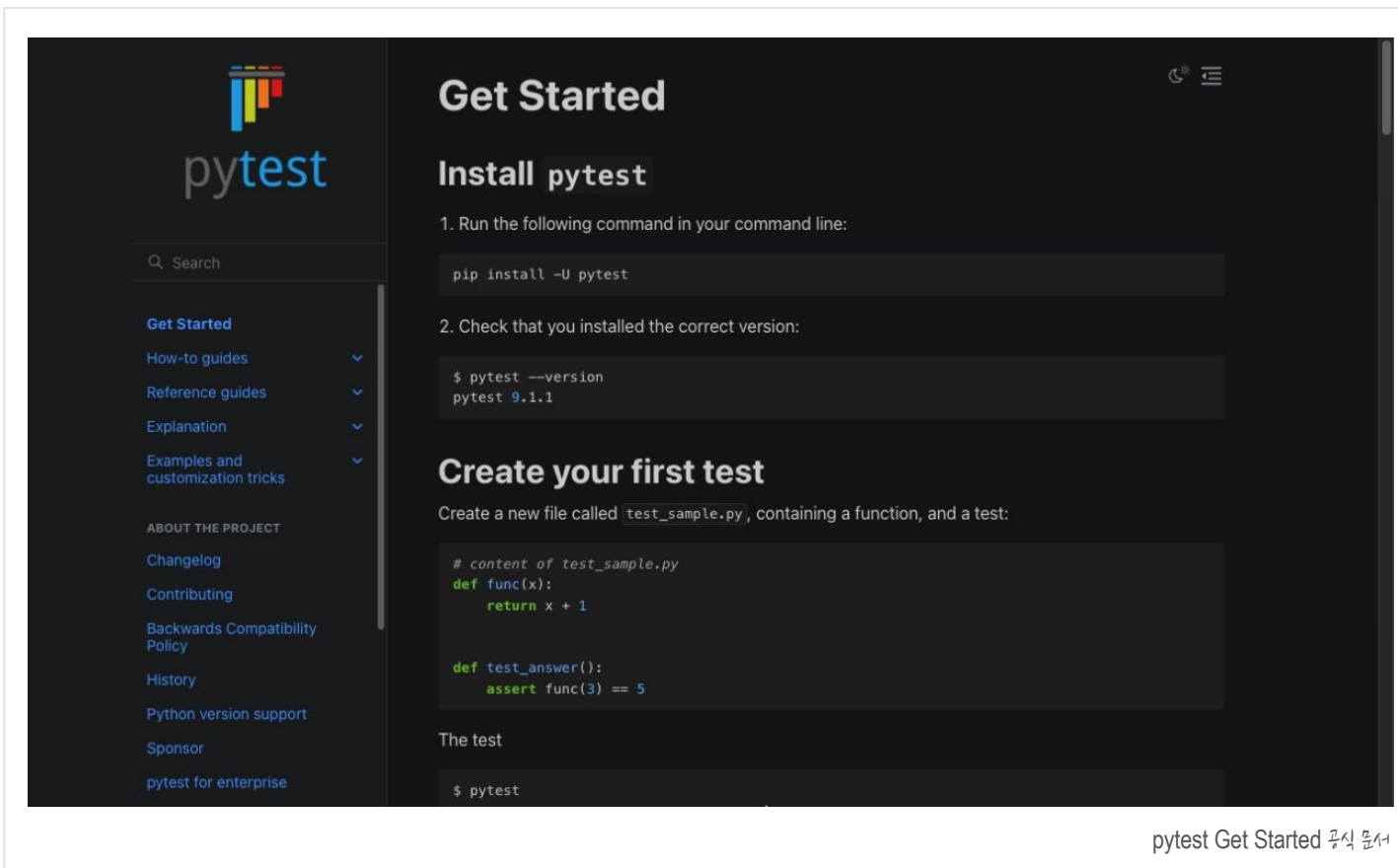
- token은 환경 변수
- 학생 필수 실습은 dry-run
- 최소 권한 확인
- 화면에 secret 노출 금지



pytest · 실패를 재현하는 가장 작은 도구

이 화면에서 꼭 볼 것

- test_ 함수 규칙
- assert로 제약 고정
- 정상·예외 함께
- 짧은 명령어로 반복



실제 실행 · 10 tests 통과

이 화면에서 꼭 볼 것

- 10개 test
- [100%] 완료
- 실행 시간 확인
- 완료 증거로 캡처

The screenshot shows the VS Code interface with the Explorer sidebar on the left displaying the project structure. The main editor area shows the file `day1_agent.py` with the following content:

```
src > day1_agent.py
1 """Day 1: a safe, dependency-free tool-calling loop
2
3 The module intentionally uses only the Python standard
4 learner run the Agent control loop before LangChain
```

Below the editor, the TERMINAL panel shows the execution of a pytest command:

```
sungjae-cha@chaseongjaeui-MacBookPro-2 llm-agent-and-workflow-aut
omation % python3 -m pytest -q
..... [100%]
10 passed in 0.05s
sungjae-cha@chaseongjaeui-MacBookPro-2 llm-agent-and-workflow-aut
omation %
```

The status bar at the bottom indicates the current file is `main*`, the encoding is `UTF-8`, the line ending is `LF`, the language is `Python`, and the Python version is `Python 3.10.19.final.0`.

VS Code 통합 터미널의 실제 pytest 결과

도구 schema · 허용 입력만

코드 전체보다 경계 한 줄을 먼저 봅니다

- 한 도구 한 책임
- 필수 인자 명시
- 설명은 모델용
- 실행 함수와 분리

```
TOOL_SCHEMAS = {  
    "read_public_text": {  
        "required": {"path": str},  
        "description": "공개 txt/md 읽기"  
    }  
}
```

예상 밖 인자를 차단

코드 전체보다 경계 한 줄을 먼저 봅니다

- 누락 검사
- 추가 필드 금지
- 실행 전 차단
- 오류 코드로 변환

```
required = schema["required"]
missing = [k for k in required if k not in args]
unexpected = set(args) - set(required)
if missing or unexpected:
    raise ToolValidationError()
```

경로 이탈을 막는 guard

코드 전체보다 경계 한 줄을 먼저 봅니다

- ../ 정규화 후 검사
- 절대경로 비교
- symlink 고려
- 테스트로 공격 재현

```
root = workspace.resolve()
candidate = (root / path).resolve()
if root != candidate and root not in candidate.parents:
    raise ToolValidationError("outside workspace")
```

오류를 공통 result로 정규화

코드 전체보다 경계 한 줄을 먼저 봅니다

- exception 경계
- 안정된 error_code
- 사용자 메시지
- trace에는 상세 stack

```
return ToolResult(  
    ok=False,  
    tool=tool_name,  
    error_code="POLICY_BLOCKED",  
    message=str(exc)  
)
```

Tool Calling Prompt의 최소 구조

코드 전체보다 경계 한 줄을 먼저 봅니다

- 역할과 금지 행동
- 입력·목표
- 도구 설명·schema
- 정상 예시 한 건

SYSTEM: 허용된 도구만 사용한다. 메일은 보내지 않는다.

TASK: 회의 문서를 읽고 할 일을 찾는다.

TOOLS: read_public_text(path: str)

OUTPUT: 필요한 경우 tool call JSON만 반환한다.

EXAMPLE:

```
{"name": "read_public_text", "arguments": {"path": "data/meeting.txt"}}
```

실습 1 · 도구 한 개를 schema로

08 MIN

강사 화면을 보며
각자 진행합니다

막히면 채팅창에
첫 오류 한 줄만
남겨주세요

- 1 업무 동사를 한 개 고른다.
- 2 필수 인자를 1-2개로 제한한다.
- 3 타임과 허용값을 적는다.
- 4 정상 호출 예시를 JSON으로 쓴다.

완료 증거 | tool schema 1개

실습 2 · 잘못된 호출 3개 만들기

08 MIN

강사 화면을 보며
각자 진행합니다

막히면 채팅창에
첫 오류 한 줄만
남겨주세요

1 unknown tool 호출을 만든다.

2 필수 인자를 하나 지운다.

3 예상 밖 인자를 하나 넣는다.

4 각 error_code를 예상한다.

완료 증거 | 실패 fixture 3개

실습 3 · workspace guard 읽기

09 MIN

강사 화면을 보며
각자 진행합니다

막히면 채팅창에
첫 오류 한 줄만
남겨주세요

1 정상 상대경로를 넣는다.

2 ../를 포함한 경로를 넣는다.

3 .py 확장자를 넣는다.

4 어느 단계에서 차단되는지 표시한다.

완료 증거 | 정상 1 + 차단 2

실습 4 · ToolResult 계약 검증

08 MIN

강사 화면을 보며
각자 진행합니다

막히면 채팅창에
첫 오류 한 줄만
남겨주세요

- 1 성공 result 필드를 적는다.
- 2 실패 result 필드를 적는다.
- 3 downstream이 exception을 몰라도 되는 이유를 쓴다.
- 4 needs_human_review 조건을 정한다.

완료 증거 | 공통 result 표

실패 1 · 모델 JSON을 바로 실행

현업에서 자주 깨지는 방식

- schema 검증 생략
- 임의 도구 이름
- 추가 인자 허용
- side effect 직행

제가 먼저 고치는 순서

- plan은 제안
- validator가 허가
- registry 함수만
- 쓰기 전 preview

실패 2 · 범용 shell 도구 제공

현업에서 자주 깨지는 방식

- rm·curl까지 가능
- 명령 문자열 해석
- 권한 범위 불명
- 평가 불가능

제가 먼저 고치는 순서

- 목적별 함수
- typed arguments
- 최소 파일 범위
- 정상·실패 test

실패 3 · 모든 오류를 retry

현업에서 자주 깨지는 방식

- 정책 오류 반복
- 비용·시간 증가
- 중복 쓰기
- 원인 은폐

제가 먼저 고치는 순서

- 오류 class 분류
- 일시 오류만 retry
- max attempts
- 중단 후 사람 검토

재직자 · Tool 권한 승인표

자동 허용

- 공개 문서 읽기
- 로컬 parsing
- schema validation
- draft 저장

사람 승인

- 메일·Issue 게시
- 고객정보 접근
- 결제·삭제
- 비공개 repo 쓰기

Gate 2 · 이 Tool Call을 실행할까요?

상황: 등록되지 않은 send_email 도구에 예상 밖의 admin=true 인자가 들어왔습니다.

1

실행·차단·사람 검토 중 무엇을 선택할까?

2

downstream에 돌려줄 error_code는 무엇일까?

3

감사 event에 어떤 요청·판단·결과를 남길까?

한국어 테이터와 PBL 사례

10:40-10:54 강의 · 10:54-11:04 Prompt·테이터 시연 · 11:04-11:25 개인 실습 · 11:25-11:30 저장

지금 배우는 내용	이 차시에서 이해할 것	함께 쟁길 내용
회의 테이터 · 담당자·기한·원문 근거	자연스러운 요약보다 근거가 남아야 한다	입력·결과·근거 예시

한국어 요약보다 먼저 근거 단위를 만듭니다

요약이 그럴듯해도 원문 근거를 찾지 못하면 업무 결과라고 부를 수 있는가?

출처

공식 원문과 이용 조건을 확인

단위

발화를 segment와 timestamp로 분리

민감성

개인정보와 회사 정보를 비식별

정답

결정·담당자·기한·근거를 연결

고정

대표 실패를 Golden Dataset에 추가

한국어 테이터를 읽는 네 가지 단위

용어	쉽게 말하면	오늘 쓰는 곳
Transcript Speech as Text	음성 내용을 시간 순서대로 문자로 옮긴 기록이다.	회의 발언과 타임스탬프를 함께 저장한다.
Segment Timed Utterance	Transcript를 시작·종료 시각이 있는 짧은 발화 단위로 나눈 것이다.	41.2-48.7초의 ‘금요일까지 초안’ 발화
Golden Dataset Reference Examples	정답 또는 검토 기준이 붙은 소량의 대표 예시 모음이다.	정상 회의·무음·모호한 기한·근거 누락 5건
Evidence Source Grounding	결론이나 Action Item이 어디에서 나왔는지 가리키는 원문 근거이다.	Action Item마다 segment_id를 연결한다.

PBL은 기술 목록이 아니라 사용자 문제에서 시작

STT·LLM·LangGraph를 모두 쓰되, 한 사용자의 한 업무 흐름에 묶는다.



- 사용자·상황
- 반복 입력
- 원하는 결정·행동

한국어 업무 데이터의 함정 | 한국어 회의록에서 가장 비싼 오류는 문법이 아니라 사람·숫자·날짜 오류다.

K-Work Copilot의 공통 입력

강사는 17분 합성 회의 전체를 보여주고, 수강생은 같은 파일의 2-3분 구간부터 검증한다.

- meeting_sample_ko_12min.wav
- timestamp transcript
- expected JSON·Dataset Card

한국어는 띄어쓰기보다 업무 맥락이 어렵다

존칭·생략·동음어·숫자·날짜·담당자 모호성을 별도 오류로 본다.

- 화자 생략
- “다음 주” 기준일
- 제품명·약어
- 책임자 추론 금지

실제 데이터처럼 보이는 최소 조건

자연스러운 맥락·역할·결정·마감이 있으면서 개인 식별정보는 없어야 한다.

- 가상 조직
- 역할명
- 합성 금액 범주
- 공개 가능한 업무 관계

한국어 업무 데이터의 함정 | 공개 데이터는 무료이지만, 사용할 수 있는 맥락까지 무료로 주지는 않는다.

데이터 공개와 사용 허가는 다르다

웹에서 보인다는 사실만으로 재배포·수업 배포가 허용되지는 않는다.

- 이용 조건
- 다운로드·신청
- 재배포 범위
- 출처 기록

Dataset Card가 실습을 재현하게 한다

출처·버전·샘플링·민감성·변환을 한 페이지에 남긴다.

- source URL
- collected_at
- license·terms
- preprocessing·limitations

Transcript schema가 STT와 LLM을 연결

텍스트만 넘기지 말고 segment·time·speaker·quality flag를 유지한다.

- start·end
- speaker
- text
- no_speech·language

Evidence가 요약의 근거를 고정한다

결정과 Action Item마다 원문 segment ID를 연결한다.

- claim
- evidence_ids
- confidence
- needs_review

한국어 업무 데이터의 함정 | Golden Dataset은 정답 모음이 아니라 팀이 합의한 품질의 경계다.

공개 데이터 선택 4단계

01

DISCOVER

공식 원자료를 찾기

02

VERIFY

이동 조건 확인

03

SAMPLE

소량·물적 결합

04

DOCUMENT

출처: 변환 기록

한국어 업무 Prompt의 기본 구조

01

ROLE

회의 기록 보조자

02

INPUT

segment·기준일

03

OUTPUT

summary·decision·action

04

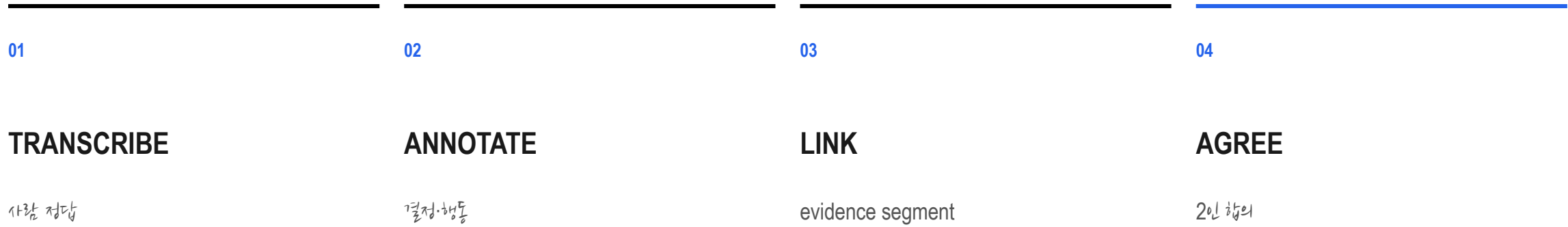
RULE

추론 금지·근거 필수

비식별 처리 순서



Golden set 만드는 순서



PBL canvas 네 칸

01

USER

누가 쓰나

02

PAIN

무엇이 누락되나

03

FLOW

AI와 사람 역경환

04

PROOF

무엇으로 좋아졌나

데이터보다 먼저 출처와 발화 구조를 확인합니다

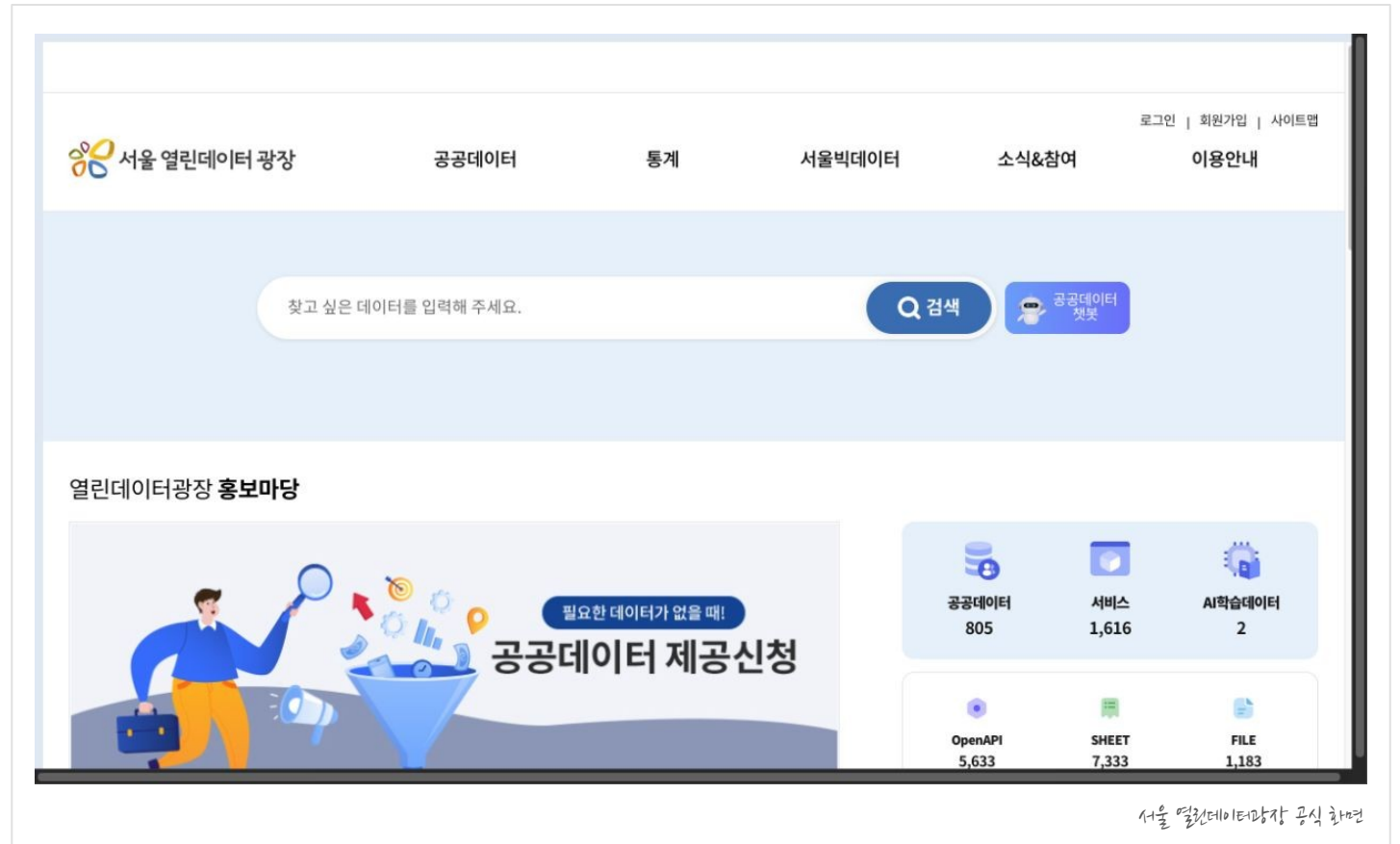
공공 데이터 → 말뭉치 → 회의록 → Managed 전사 화면

도구·서비스	도메인·역할	이번 강의에서 쓰는 이유	유사 도구와의 차이
서울 열린데이터광장	공공·행정 데이터	한국어 공개 데이터의 출처와 갱신 주기를 확인	회의 발화와 데이터는 별도 가공 필요
모두의 말뭉치	한국어 원자문	문장·대화 자료의 언어 특성을 이해	이용 신청과 재배포 조건 확인
국회 회의록	공식 회의 발화	화자·안전·발언 구조가 실제 업무와 가까움	목표 업무에 맞는 정답 라벨은 없음
Google Meet	Managed STT·노트	현업에 이미 있는 전사·요약 기능을 활용	구독·edition·관리자 정책 의존

서울 열린데이터광장 · 한국어 공공 데이터 탐색

이 화면에서 꼭 볼 것

- 공식 제공처를 먼저
- 검색어·제공기관 기록
- 다운로드 형식 확인
- 소량 샘플부터



국립국어원 모두의 말뭉치 · 한국어 원자료

이 화면에서 꼭 볼 것

- 신청·이용 조건 확인
- 말뭉치 종류와 목적 구분
- 원문 재배포 주의
- dataset card에 출처

언어정보나눔터

모두의 말뭉치

말뭉치 분류

배포 연도

국립국어원 모두의 말뭉치 화면

국회 회의록 빅데이터 · 실제 발화 구조

이 화면에서 꼭 볼 것

- 발언자·회의 맥락
- 공개 회의 기록 사례
- 페이지·첨부 공개 범위 확인
- 학습용 소량 사례

SNS 로그인

NXfG

지식과 정보가
나비처럼 자유로운 세상

NAIL
National Assembly AI Library

국회도서관
발언 빅데이터

검색어를 입력하세요. (발언자명, 발언 내용, 안건명, 회의명)

상세검색

Q

국회·지방의회 회의록에 대한 발언자, 키워드 검색 및 다운로드, 인포그래픽 등
다양한 의정기록 데이터 서비스를 제공합니다.

회의록 건수

39,370 건

발언 데이터셋

16,637,220 건

서비스 통계

최근 회의록

회의록수 | 발언 데이터셋

[임시] 제22대 국회 제438회 제3차 교육위... 2026-08-14

국회 회의록 빅데이터 공식 화면

회의 segment schema

코드 전체보다 경계 한 줄을 먼저 봅니다

- timestamp 유지
- speaker는 모르면 null
- 원문 text 보존
- 품질 flag 별도

```
segment = {  
  "id": "s12",  
  "start": 41.2,  
  "end": 48.7,  
  "speaker": "역할1",  
  "text": "금요일까지 초안을...",  
  "quality_flags": []  
}
```

Action Item에 evidence 연결

코드 전체보다 경계 한 줄을 먼저 봅니다

- task·owner·date
- 근거 segment
- confidence 정책
- 추론 시 review

```
action = {  
  "task": "FAQ 30건 정리",  
  "owner": "역할1",  
  "due_date": "2026-08-27",  
  "evidence_ids": ["s12"],  
  "confidence": 0.92  
}
```

모호한 날짜는 추론하지 않는다

코드 전체보다 경계 한 줄을 먼저 봅니다

- 기준일 필요
- null 허용
- 모호성 코드
- 사람에게 질문

```
if due_text in ["다음 주", "조만간"]:  
    return {  
        "due_date": None,  
        "needs_review": True,  
        "reason": "RELATIVE_DATE"  
    }
```

한국어 회의 Prompt · 모호하면 추론하지 않기

코드 전체보다 경계 한 줄을 먼저 봅니다

- 역강제를 한 문장으로
- 금지 행동을 먼저
- 출력 schema 고정
- 원문 근거 요구

```
SYSTEM: 한국어 회의 기록 보조자
RULES:
- 원문에 없는 담당자·기한을 추론하지 않는다.
- 모호하면 null과 needs_review를 반환한다.
INPUT: segments[] + meeting_date
OUTPUT: MeetingResult JSON
EVIDENCE: 모든 결정·할 일에 evidence_ids를 연결한다.
```

Dataset card 최소 필드

코드 전체보다 경계 한 줄을 먼저 봅니다

- 출처
- 수집 시점
- 민감성
- 변환·제거

```
dataset:  
  source: official_url  
  collected_at: 2026-08-22  
  purpose: classroom_demo  
  contains_pii: false  
  transformations: [sampling, anonymize]  
  limitations: [synthetic_speakers]
```

실습 1 · 한국어 사례 후보 3개

08 MIN

강사 화면을 보며
각자 진행합니다

막히면 채팅창에
첫 오류 한 줄만
남겨주세요

- 1 공식 데이터 사이트에서 검색어를 정한다.
- 2 사례 후보를 세 개 적는다.
- 3 업무 흐름과 가장 가까운 한 개를 고른다.
- 4 선택 이유를 한 문장으로 쓴다.

완료 증거 | 후보 3개 + 선택 1개

실습 2 · 이용 조건 확인표

08 MIN

강사 화면을 보며
각자 진행합니다

막히면 채팅창에
첫 오류 한 줄만
남겨주세요

- 1 제공 기관과 URL을 기록한다.
- 2 다운로드·신청 필요 여부를 적는다.
- 3 재배포 가능 여부를 확인한다.
- 4 불확실하면 강사 fixture를 선택한다.

완료 증거 | source checklist

실습 3 · 상세 회의에서 근거 구간 찾기

10 MIN

강사 화면을 보며
각자 진행합니다

막히면 채팅창에
첫 오류 한 줄만
남겨주세요

- 1 meeting_sample_ko_12min.txt를 엽니다.
- 2 결정 1개와 정정 발언 1개를 찾습니다.
- 3 담당·기한 Action Item 2개를 찾습니다.
- 4 “다음 주 중”을 needs_review로 표시합니다.

완료 증거 | evidence segment 4개

실습 4 · Golden JSON과 비교

10 MIN

강사 화면을 보며
각자 진행합니다

막히면 채팅창에
첫 오류 한 줄만
남겨주세요

- 1 내가 찾은 title·decision을 구조화합니다.
- 2 Action Item 두 개에 evidence ID를 연결합니다.
- 3 meeting_sample_ko_12min_expected.json과 비교합니다.
- 4 다른 부분을 오류·허용 표현 차이로 나눕니다.

완료 증거 | 내 JSON + 기존 JSON diff

실패 1 · 테이터가 너무 크다

현업에서 자주 깨지는 방식

- 수십 시간 audio
- 수백 MB 전체 말뭉치
- 전처리만 수업 소진
- 정답 만들기 불가

제가 먼저 고치는 순서

- 2-3분 audio
- 10-30건 text
- golden 3-5건
- 확장 과제는 별도

실패 2 · 공개 페이지를 그대로 재배포

현업에서 자주 깨지는 방식

- 이용 조건 미확인
- 첨부 전체 공유
- 출처 누락
- 신청 계정 공유

제가 먼저 고치는 순서

- 공식 링크 제공
- 허용된 소량 샘플
- 출처·날짜 기록
- fixture 대체

실패 3 · 정답 없이 prompt를 튜닝

현업에서 자주 깨지는 방식

- 좋아 보이는 출력
- 기준 매번 변경
- 모델 비교 불가
- 오류 회피 모름

제가 먼저 고치는 순서

- golden example
- rubric
- 고정 dataset
- experiment 비교

제작자 · 실제 업무를 안전하게 추상화

가져오지 않을 것

- 실명·고객번호
- 계약 금액
- 회사 URL
- 내부 전략

남길 관계

- 역할 간 의사결정
- 업무 단계
- 승인 조건
- 실패 비용

Gate 3 · **이 Action Item을 믿어도 될까요?**

상황: 요약에는 “다음 주에 민지가 배포”라고 적혔지만 원문에는 담당자와 날짜가 명확하지 않습니다.

1 어떤 segment를 evidence로 다시 확인할까?

2 담당자·기한을 null로 둘지 사람에게 물을지 결정하자.

3 이 사례를 Golden Dataset에 어떤 실패 유형으로 남길까?

12:00-12:50

1일자 4차시

Python·VS Code·Git 환경

12:00-12:10 강의 · 12:10-12:25 강사 세팅 시연 · 12:25-12:45 개인 실습 · 12:45-12:50 확인

지금 배우는 내용	이 차시에서 이해할 것	함께 챙길 내용
Python 경로 · Interpreter · Git branch·PR · Codex review	설치보다 같은 명령으로 다시 실행되는 것이 중요하다	버전·경로·Draft PR 또는 local diff

같은 명령이 같은 Python을 가리켜야 합니다

설치가 실패해도 수업과 프로젝트를 계속할 수 있는 두 번째 경로가 있는가?

확인

OS·Python·Git 버전을 먼저 캡처

격리

프로젝트 .venv를 생성

선택

VS Code와 Notebook interpreter를 맞춤

보호

secret·대용량 파일을 Git 밖에 둬

복구

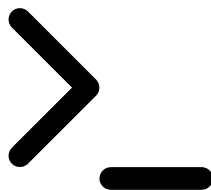
첫 commit과 fixture 경로를 남김

설치보다 먼저 구분할 네 가지 환경

용어	쉽게 말하면	오늘 쓰는 곳
Runtime Execution Environment	코드가 실제로 실행되는 Python·운영체제·패키지 환경이다.	python3 --version과 실행 경로를 먼저 확인한다.
Interpreter Selected Python	VS Code나 Notebook이 사용할 특정 Python 실행 파일이다.	.venv/bin/python을 VS Code Interpreter로 선택한다.
Virtual Environment Isolated Packages	프로젝트별 Python 패키지를 격리하는 폴더이다.	python3 -m venv .venv 후 같은 환경에서 설치·실행한다.
Repository Versioned Project Folder	코드·문서·데이터와 변경 이력을 함께 관리하는 프로젝트 폴더이다.	src·tests·data·materials 폴더를 하나의 저장소로 관리한다.

무료의 정의를 먼저 합의한다

학생은 외부 LLM API 비용 없이 필수 실습을 완주할 수 있어야 한다.



- Python·Git·VS Code
- fixture·local LLM
- 선택 서비스 초과 과금 주의

수업을 살리는 기준 | 환경 문제의 절반은 패키지가 아니라 '지금 어떤 Python을 쓰는가'에서 시작한다.

필수 경로와 강사 가속 경로를 분리

학생은 로컬·결정론적 코드, 강사는 Codex·Claude live demo를 사용한다.

- 필수: 직접 실행
- 강사: 생성·수정 가속
- 공통: diff·test 검토

Python은 실행기, VS Code는 작업 공간

extension이 Python 자체를 설치해 주는 것은 아니다.

- Python interpreter
- VS Code editor
- Python·Jupyter extension

가상환경은 프로젝트의 재현 경계

전역 환경 대신 프로젝트별 dependency를 격리한다.

- `python -m venv`
- interpreter 선택
- `requirements·pyproject`

수업을 살리는 기준 | 첫 commit은 저장이 아니라 복구 지점을 만드는 일이다.

Git과 GitHub는 변경의 증거와 검토 장소

작은 commit을 branch와 PR로 연결하면 무엇을 바꿨고 누가 확인했는지가 남는다.

- status·diff
- branch·commit
- push·draft PR
- review·merge

Jupyter는 설명과 실행의 다리

한 셀 한 개념으로 결과를 보고, 모듈 코드는 src에 둔다.

- Markdown ^{설명}
- Code ^{실행}
- output ^{결과}
- Restart & Run All

Secret은 코드·노트북·캡처 밖에 둔다

실제 값은 환경 변수, 저장소에는 이름만 있는 example을 둔다.

- .env.example
- .gitignore
- 화면 비식별

Harness Engineering은 PR에서 완료된다

Spec·히용 파일·test·review rule을 저장소에 두고 Codex 결과를 사람이 merge 전에 확인한다.

- AGENTS.md
- small diff
- CI-unit test
- Codex review 초안

수업을 살리는 기준 | Harness Engineering의 출발은 좋은 프롬프트보다 깨끗한 작업공간이다.

환경 설치 순서

01

PYTHON

버전 확인

02

VSCODE

extension

03

GIT

identity

04

REPO

clone·open

가상환경 준비

01

CREATE

python -m venv

02

ACTIVATE

OS^별 명령

03

INSTALL

requirements

04

VERIFY

python -m pip

VS Code interpreter 선택

01

PALETTE

Command Palette

02

SELECT

Python interpreter

03

KERNEL

notebook kernel

04

CHECK

sys.executable

GitHub branch와 Draft PR 준비

01

BRANCH

작은 목적 한 개

02

DIFF

변경 범위 확인

03

PUSH

교육용 remote

04

DRAFT PR

목표·test·위험

Codex PR 검토 루프



설치 화면은 ‘어디서 받는가’보다 ‘무엇을 확인하는가’가 중요합니다

Python → VS Code interpreter → Git identity → sandbox repository

01	내 OS에 맞는 runtime 선택	버전·실행 경로
02	editor가 쓸 Python 지정	.venv interpreter
03	Git 작성자와 기본 상태 확인	identity·status
04	교육용 저장소만 연결	sandbox remote

Python 공식 다운로드 · interpreter 준비

이 화면에서 꼭 볼 것

- OS와 버전 확인
- 설치 후 새 terminal
- python3 --version
- 회사 PC는 권한 확인

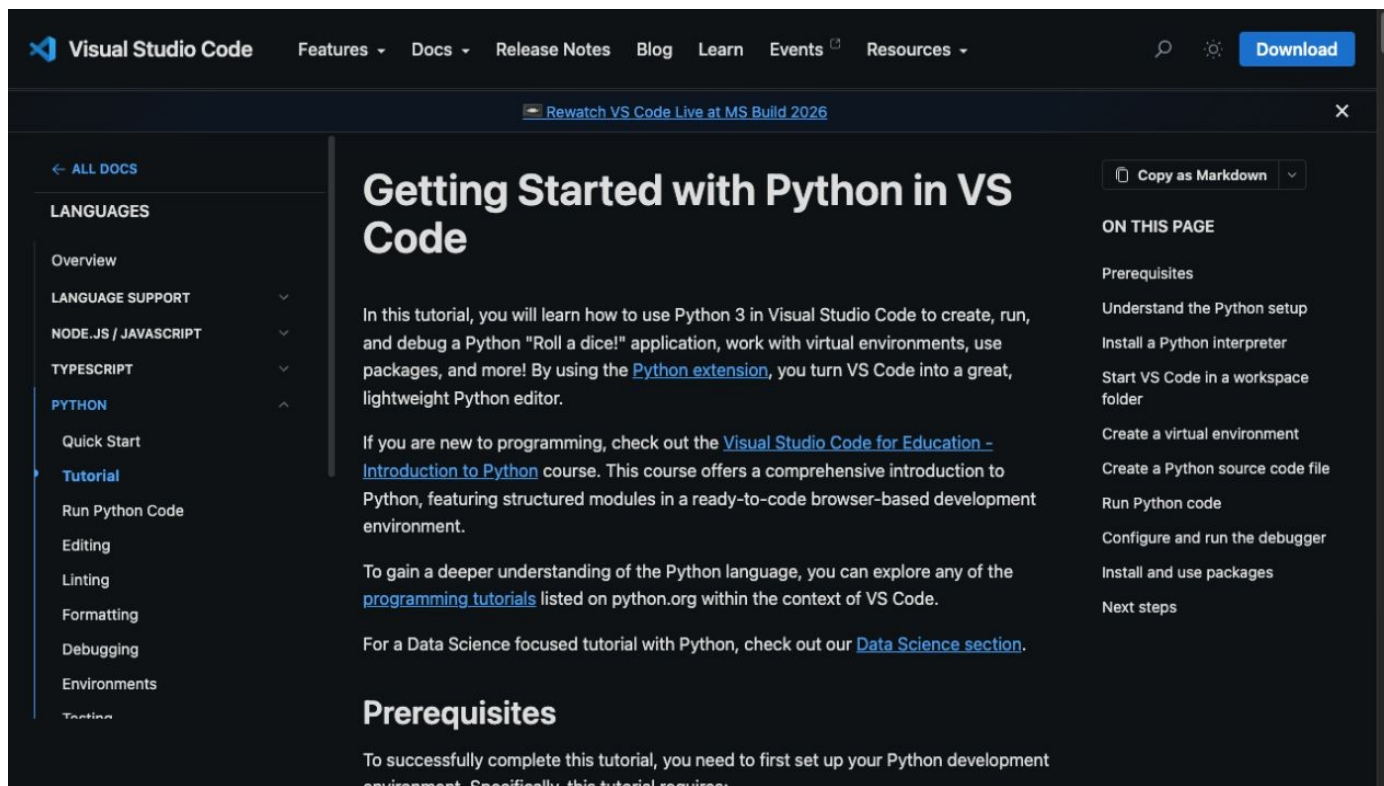
The screenshot shows the Python.org website. At the top, there's a navigation bar with links: Python, PSF, Docs, PyPI, Jobs, and Community. Below this is a header with the Python logo, a 'Donate' button, a search bar, and a 'Socialize' button. A secondary navigation bar contains links: About, Downloads, Documentation, Community, Success Stories, News, and Events. The main content area features a large banner with the text 'Download the latest version of Python' and a button 'Download Python 3.14.7'. Below this, there are links for different operating systems: Windows, Linux/Unix, macOS, Android, iOS, and other. There's also a link for pre-releases and Docker images. To the right of the text is an illustration of two parachutes carrying boxes. Below the banner, there's a section titled 'Active Python releases' with a link to the Python Developer's Guide. This section contains a timeline for Python 2.7 and Python 3.6, both marked as 'end-of-life'.

Python.org Downloads 공식 화면

VS Code Python 튜토리얼 · 세 구성요소

이 화면에서 꼭 볼 것

- VS Code
- Python extension
- Python interpreter
- virtual environment

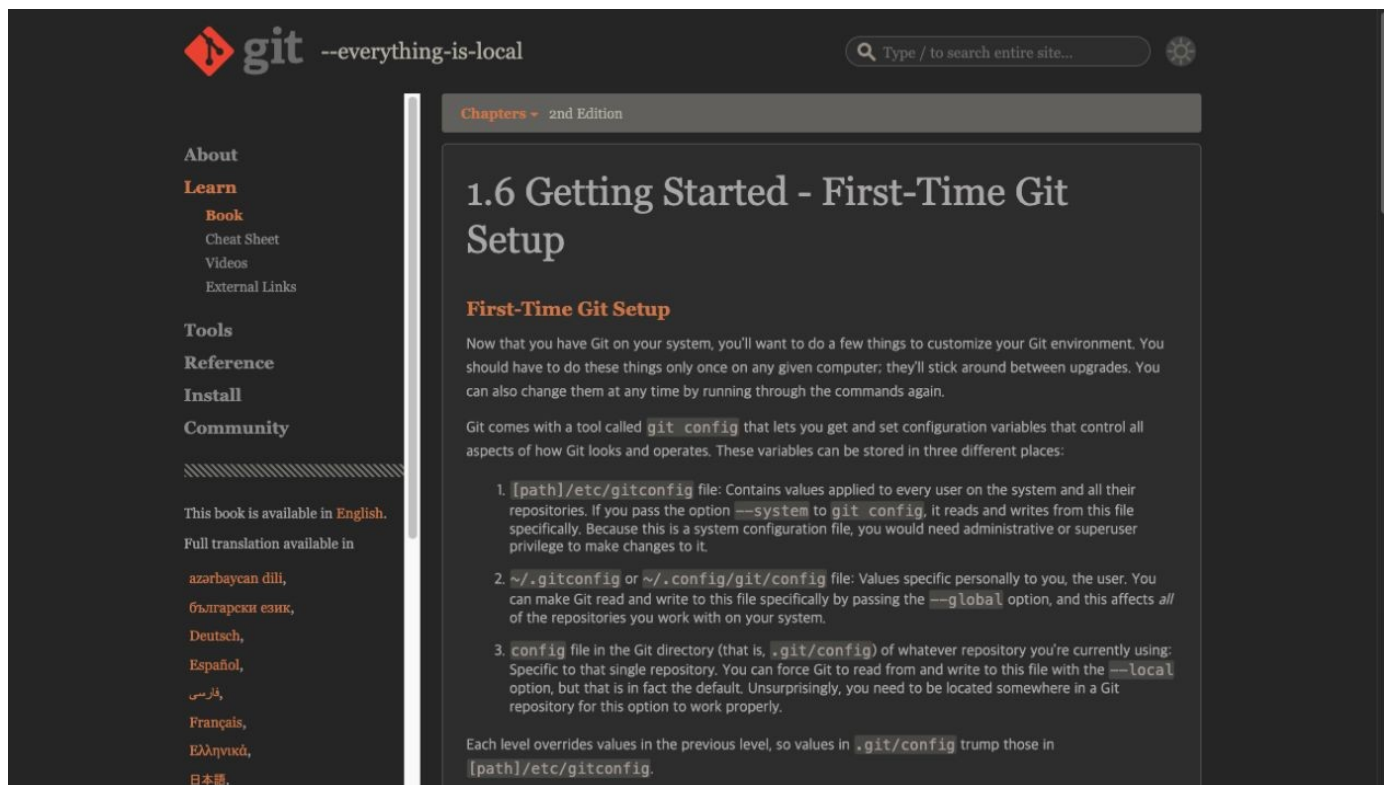


VS Code Python Tutorial 공식 화면

Git 첫 설정 · identity와 기본 확인

이 화면에서 꼭 볼 것

- `git --version`
- `user.name·user.email`
- 회사 계정과 교육 계정 구분
- 설정 범위 확인

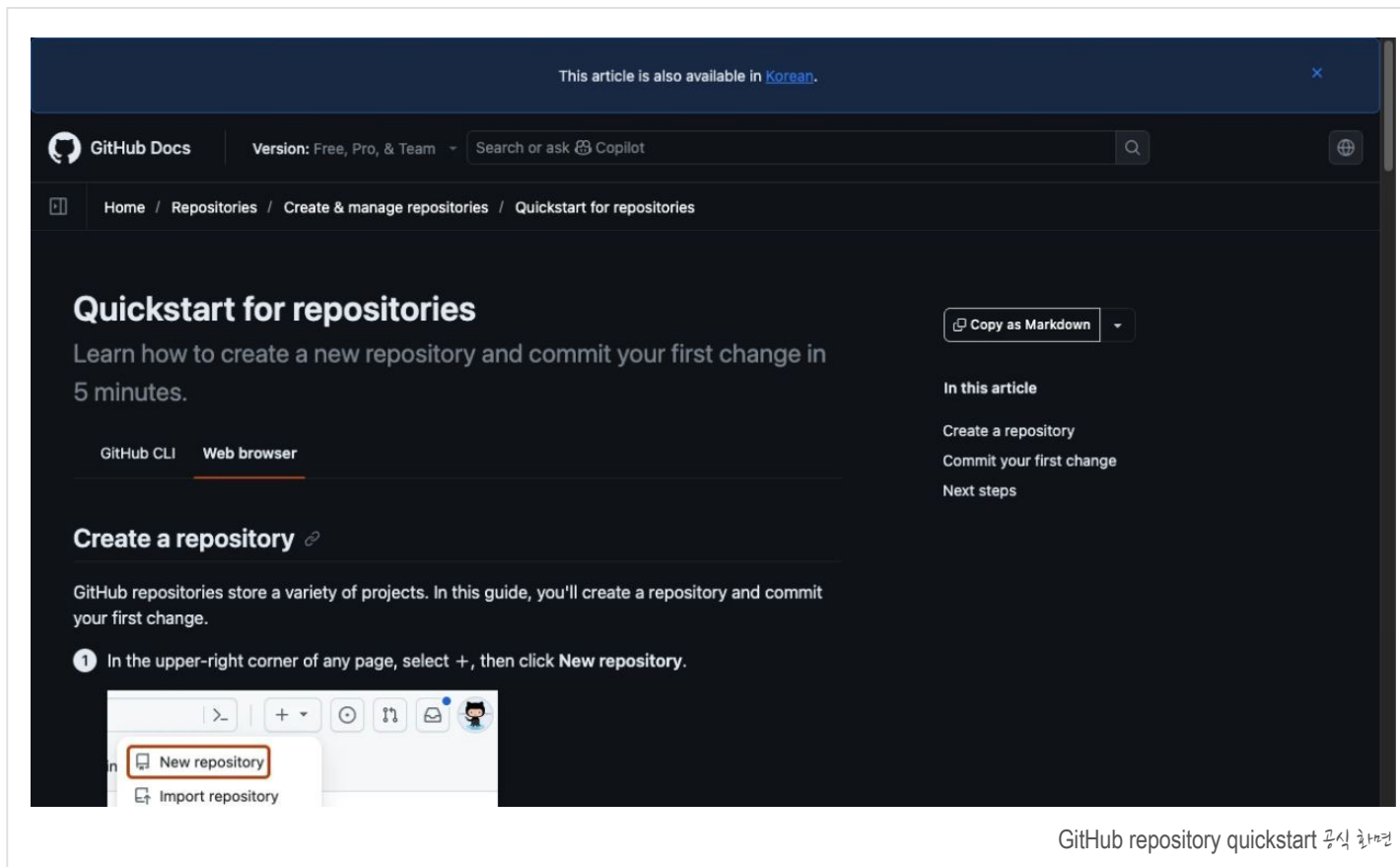


Pro Git First-Time Git Setup

GitHub repository quickstart · 교육용 sandbox

이 화면에서 꼭 볼 것

- 공개/비공개 선택 주의
- README·.gitignore
- 교육용 repo
- 비공개 회사 코드 금지



환경 확인 네 줄

코드 전체보다 경계 한 줄을 먼저 봅니다

- 명령어와 기대 결과
- PATH 문제 확인
- 새 terminal 재시작
- 캡처에 개인정보 최소화

```
python3 --version  
git --version  
code --version  
python3 -m pip --version
```

가상환경 명령 · macOS/Linux

코드 전체보다 경계 한 줄을 먼저 봅니다

- .venv 생성
- 활성화
- dependency 설치
- interpreter 경로 확인

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements-day1.txt
python -c "import sys; print(sys.executable)"
```

가상환경 명령 · Windows PowerShell

코드 전체보다 경계 한 줄을 먼저 봅니다

- ExecutionPolicy 오류 분리
- py launcher
- 같은 requirements
- 경로 화면 확인

```
py -3 -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install -r requirements-day1.txt
python -c "import sys; print(sys.executable)"
```

작은 branch에서 Draft PR까지

코드 전체보다 경계 한 줄을 먼저 봅니다

- 교육용 repo만
- 관련 파일만 stage
- test·diff 증거
- gh가 없으면 GitHub 화면

```
git switch -c test/reject-python-file
python3 -m pytest -q
git diff --check
git add tests/test_day1_agent.py
git commit -m "test: reject python file reads"
git push -u origin HEAD
gh pr create --draft --fill
```

AGENTS.md · Codex가 볼 Review 기준

코드 전체보다 경계 한 줄을 먼저 봅니다

- 저장소 특화 기준
- 결과 중심 규칙
- 기계 검사는 CI
- 최종 판단은 사람

Code Review Rules

Safety boundary

- workspace 밖 파일 접근을 허용하지 않는다.
- 외부 쓰기는 사람 승인 없이 실행하지 않는다.

Tests

- 동작 변경에는 정상·실패 unit test를 함께 둔다.

실습 1 · 버전 확인 캡처

08 MIN

강사 화면을 보며
각자 진행합니다

막히면 채팅창에
첫 오류 한 줄만
남겨주세요

- 1 새 terminal을 연다.
- 2 Python·Git 버전을 확인한다.
- 3 VS Code가 같은 폴더를 열었는지 확인한다.
- 4 실패 항목을 표에 표시한다.

완료 증거 | 버전 3개 + 상태표

실습 2 · 가상환경과 interpreter

10 MIN

강사 화면을 보며
각자 진행합니다

막히면 채팅창에
첫 오류 한 줄만
남겨주세요

- 1 .venv를 만든다.
- 2 가상환경을 활성화한다.
- 3 VS Code interpreter를 선택한다.
- 4 sys.executable을 출력한다.

완료 증거 | 가상환경 경로 출력

실습 3 · notebook 열기

08 MIN

강사 화면을 보며
각자 진행합니다

막히면 채팅창에
첫 오류 한 줄만
남겨주세요

1 materials/day1 notebook을 연다.

2 선택한 kernel을 확인한다.

3 첫 Markdown·Code 셀을 실행한다.

4 output이 남는 위치를 확인한다.

완료 증거 | 첫 셀 성공 화면

실습 4 · Branch→Codex→Test→PR 준비

12 MIN

강사 화면을 보며
각자 진행합니다

막히면 채팅창에
칭 오류 한 줄만
남겨주세요

- 1 교육용 branch를 만들고 변경 목표를 네 줄로 적습니다.
- 2 Codex에는 tests/test_day1_agent.py 한 파일만 허용합니다.
- 3 생성된 diff를 직접 읽고 pytest를 실행합니다.
- 4 GitHub 연결이 되면 Draft PR, 아니면 local diff를 저장합니다.

예시 | Objective: .py 읽기 차단 test 추가 / Scope: tests/test_day1_agent.py / Do not: src 변경 / Verify: python3 -m pytest -q

완료 증거 | branch + test + Draft PR 또는 local diff

실패 1 · 설치를 끝없이 붙잡는다

현업에서 자주 깨지는 방식

- 한 명의 PATH에 30분
- 전체 대기
- 학습 목표 상실
- 다운로드 경쟁

제가 먼저 고치는 순서

- 10분 timebox
- checkpoint zip
- fixture lane
- 점심 후 재합류

실패 2 · 전역 Python에 모두 설치

현업에서 자주 깨지는 방식

- 버전 충돌
- 권한 오류
- 재현 불가
- 다른 프로젝트 영향

제가 먼저 고치는 순서

- .venv
- python -m pip
- requirements 고정
- interpreter 확인

실패 3 · Agent에게 저장소 전체 수정 허용

현업에서 자주 깨지는 방식

- 범위 모호
- 불필요 파일 변경
- test 삭제
- secret 접근

제가 먼저 고치는 순서

- 허용 파일
- 금지 조건
- diff 우선
- test 결과 요구

재직자 · 회사 PC 제약 lane

제약

- 관리자 권한 없음
- 외부 다운로드 차단
- GitHub 접속 제한
- GPU 없음

대체

- portable/승인 절차
- 사전 제공 fixture
- 로컬 Git
- 작은 CPU 모델·텍스트

Gate 4 · 다른 PC에서도 다시 실행할 수 있을까요?

상황: 저장소는 받았지만 .venv가 없고 Notebook이 전역 Python을 보고 있습니다.

1 첫 세 명령으로 runtime과 경로를 어떻게 확인할까?

2 10분 안에 설치가 안 되면 어떤 fixture로 계속할까?

3 정상 상태를 어떤 commit과 캡처로 남길까?

안전한 Agent 실행 루프

12:50-13:00 강의 · 13:00-13:10 코드 시연 · 13:10-13:35 개인 실습 · 13:35-13:40 확인

지금 배우는 내용	이 차시에서 이해할 것	함께 챙길 내용
Planner · Validator · Executor · Test	LLM 전에 정상과 실패를 규칙으로 검증한다	정상·실패 test 결과

LLM을 붙이기 전에 실패 상태에서 고정합니다

이 실행 루프는 언제 재시도하고, 언제 사람에게 넘기고, 언제 완전히 멈추는가?

계획

규칙 기반 planner로 next action 선택

검사

validator가 schema와 policy 확인

실행

executor가 줬은 tool만 호출

관찰

ToolResult로 성공·실패를 정규화

종단

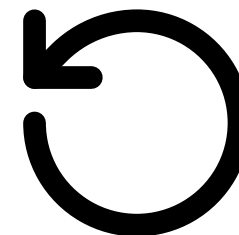
timeout·attempt·terminal status로 종료

Agent 실행 루프를 나누는 네 역할

용어	쉽게 말하면	오늘 쓰는 곳
Planner Next-action Selector	현재 요청을 보고 다음에 사용할 도구와 인자를 정하는 부분이다.	‘txt를 읽어줘’에서 read_public_text와 path를 추출한다.
Executor Validated Runner	Planner의 요청을 다시 검사한 뒤 실제 도구 함수를 실행하는 부분이다.	Registry에 있는 허용 도구만 호출한다.
Timeout Maximum Wait	도구가 응답하지 않을 때 기다릴 최대 시간이다.	10초를 넘으면 timeout_error로 기록하고 fallback을 선택한다.
Idempotency Same Request, One Effect	같은 요청을 여러 번 실행해도 외부 결과가 한 번만 생기게 하는 성질이다.	request_id가 이미 성공했으면 저장된 결과를 반환한다.

처음부터 LLM을 넣지 않는 이유

**결정론적 planner로 제어 계약을 먼저 테스트하면
모델 변동과 코드 오류를 분리할 수 있다.**



- 규칙 기반 plan
- 고정 fixture
- 같은 executor contract

운영에서 먼저 터지는 곳 | 무한 retry는 회복력이 아니라 장애를 늦게 발견하는 방식이다.

Planner는 “무엇을 할까”만 결정

도구 이름과 인자를 제안하고 실행 권한은 갖지 않는다.

- input parse
- planned_call
- no side effect

Validator는 안전 경계를 고정

허용 도구·필수 인자·타입·추가 필드·경로를 검사한다.

- schema
- policy
- path guard
- error code

Executor는 등록 함수만 호출

동적 eval이나 임의 import 없이 registry에서 함수 객체를 찾는다.

- registry lookup
- kwargs
- exception boundary

운영에서 먼저 터지는 곳 | 결정론적 planner로 통과하지 못한 계약은 LLM을 넣어도 안정되지 않는다.

ToolResult는 성공과 실패의 공통 언어

downstream graph가 traceback 구조를 몰라도 다음 행동을 고를 수 있다.

- ok
- data
- error_code
- message

read_public_text는 의도적으로 좁다

workspace 안의 공개 txt·md만 읽는 도구로 최소 권한을 체험한다.

- root boundary
- suffix allowlist
- UTF-8

count_action_markers는 LLM 전 fixture

한국어 행동 표현을 규칙으로 찾아 구조화 파이프라인을 먼저 연결한다.

- 까지·작성·검토
- evidence line
- deterministic

운영에서 먼저 터지는 곳 | Agent의 실력은 성공 횟수보다 실패 상태를 얼마나 잘 구분하는지에서 드러난다.

코드 읽기 순서

01

DATA

dataclass·schema

02

POLICY

validate·guard

03

EXECUTE

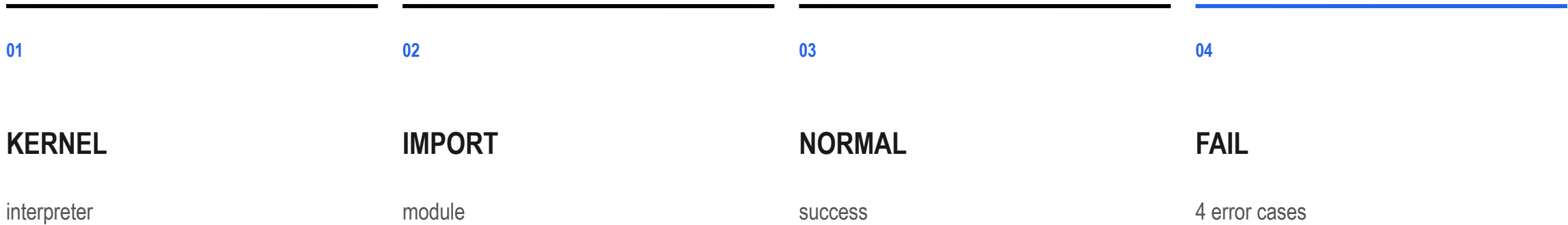
registry·result

04

DEMO

__main__

Run All 순서



실패 주입 순서

01

UNKNOWN

도구 없음

02

MISSING

인자 누락

03

TRAVERSAL

경로 이탈

04

DUPLICATE

cache 확인

pytest 빨강→초록

01

RUN

현재 실패

02

READ

첫 오류

03

FIX

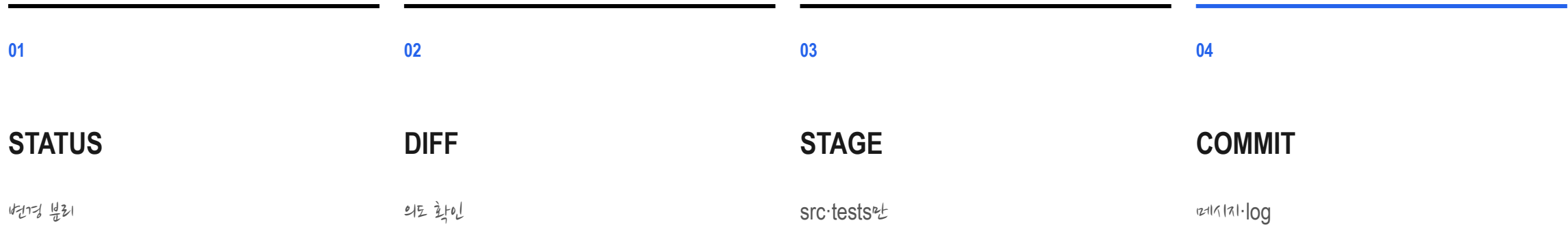
최소 변경

04

RERUN

전체 10개

Git checkpoint 만들기



이 네 화면은 실행보다 복구 경로를 확인하기 위한 것입니다

Notebook 실행기 → interpreter → Agent 수정 → 같은 repo 검토

01	Notebook kernel을 준비	Run All 가능
02	Terminal과 Notebook Python 맞추기	같은 interpreter
03	작은 spec으로 한 경계만 수정	제한된 diff
04	다른 Agent로 실패 조건 검토	test 재실행

Jupyter 설치 · notebook 실행기의 공식 경로

이 화면에서 꼭 볼 것

- JupyterLab·Notebook 구분
- VS Code extension과 kernel
- 프로젝트 가상환경 사용
- Run All이 완료된 증거

The Jupyter Foundation [call for funding proposals](#) is now open. [Submit a proposal](#) for improving Jupyter.

jupyter

Try **Install** Community Subprojects ▾ AI Documentation News Social Governance Security About

Installing Jupyter

Get up and running on your computer

Project Jupyter's tools are available for installation via the **Python Package Index**, the leading repository of software created for the Python programming language.

This page uses instructions with **pip**, the recommended installation tool for Python. If you require *environment management* as opposed to just installation, look into **conda**, **mamba**, **pipenv**, and **Homebrew**.

JupyterLab

Install JupyterLab with **pip**:

```
pip install jupyterlab
```

Note: If you install JupyterLab with conda or mamba, we recommend using the **conda-forge channel**.

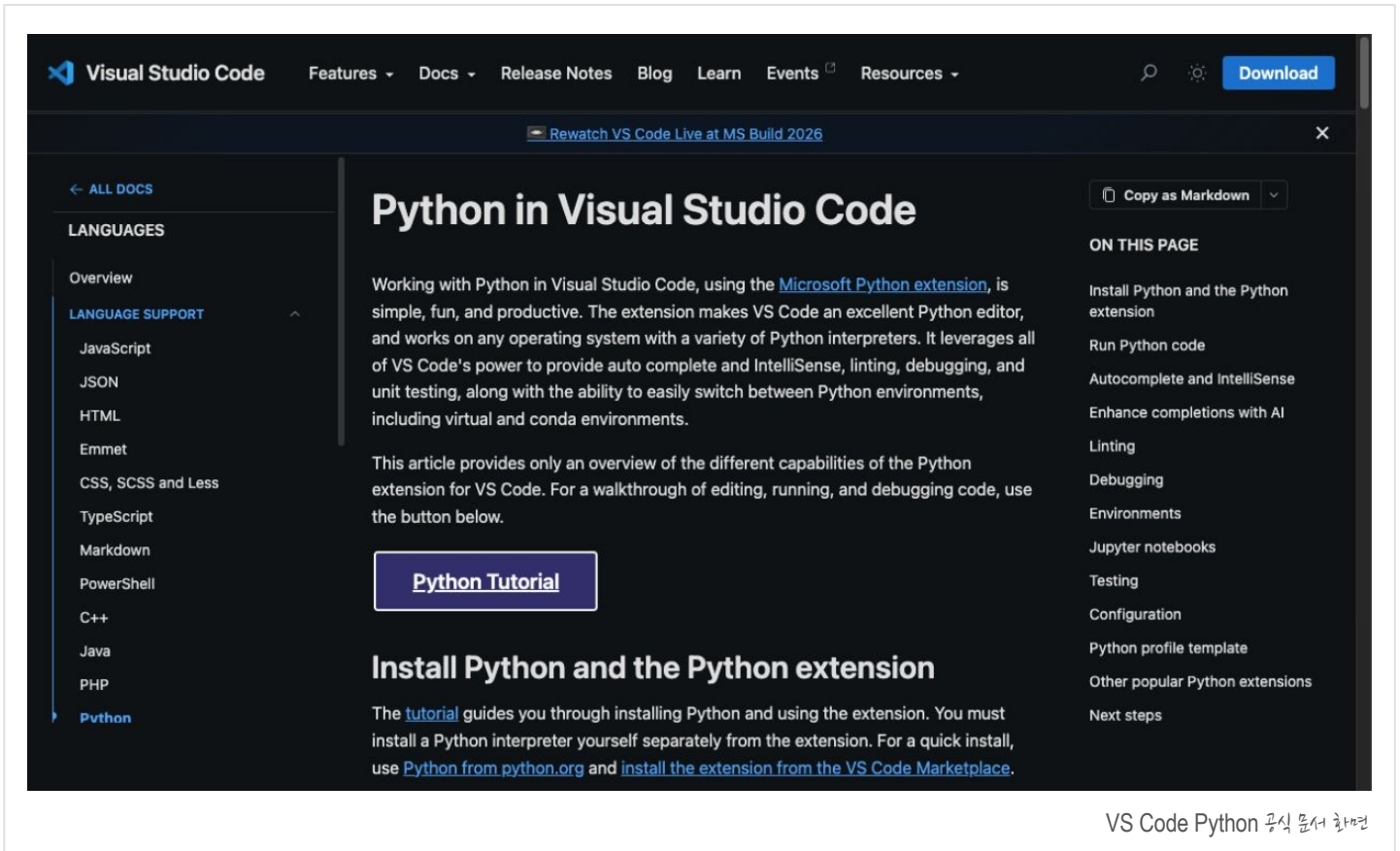
Once installed, launch JupyterLab with:

Jupyter Install 공식 화면

VS Code Python · editor와 interpreter 연결

이 화면에서 꼭 볼 것

- Python extension
- interpreter 표시
- Run Python File
- testing·debugging 연결

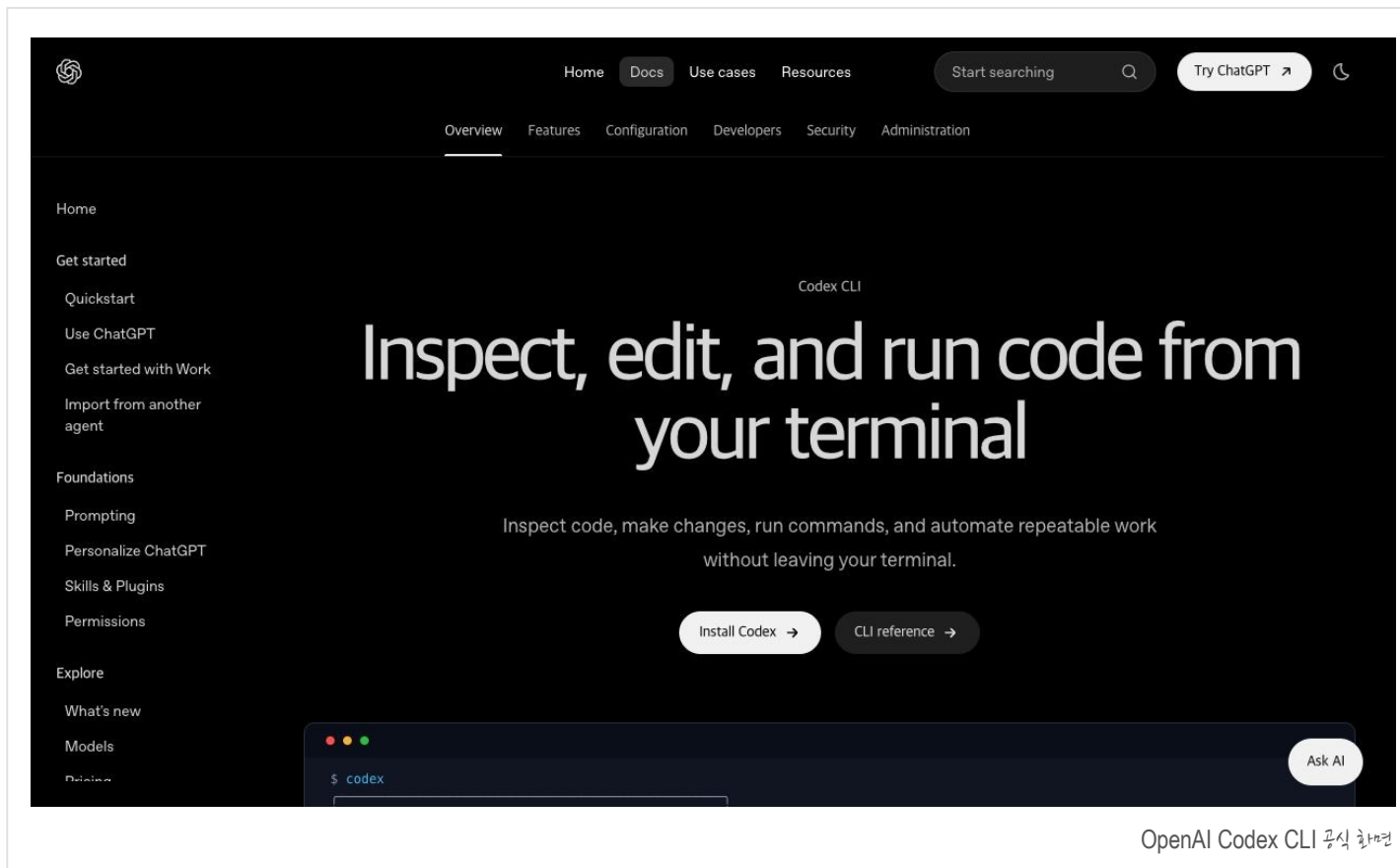


VS Code Python 공식 문서 화면

Codex CLI · 작은 spec으로 수정 가속

이 화면에서 꼭 볼 것

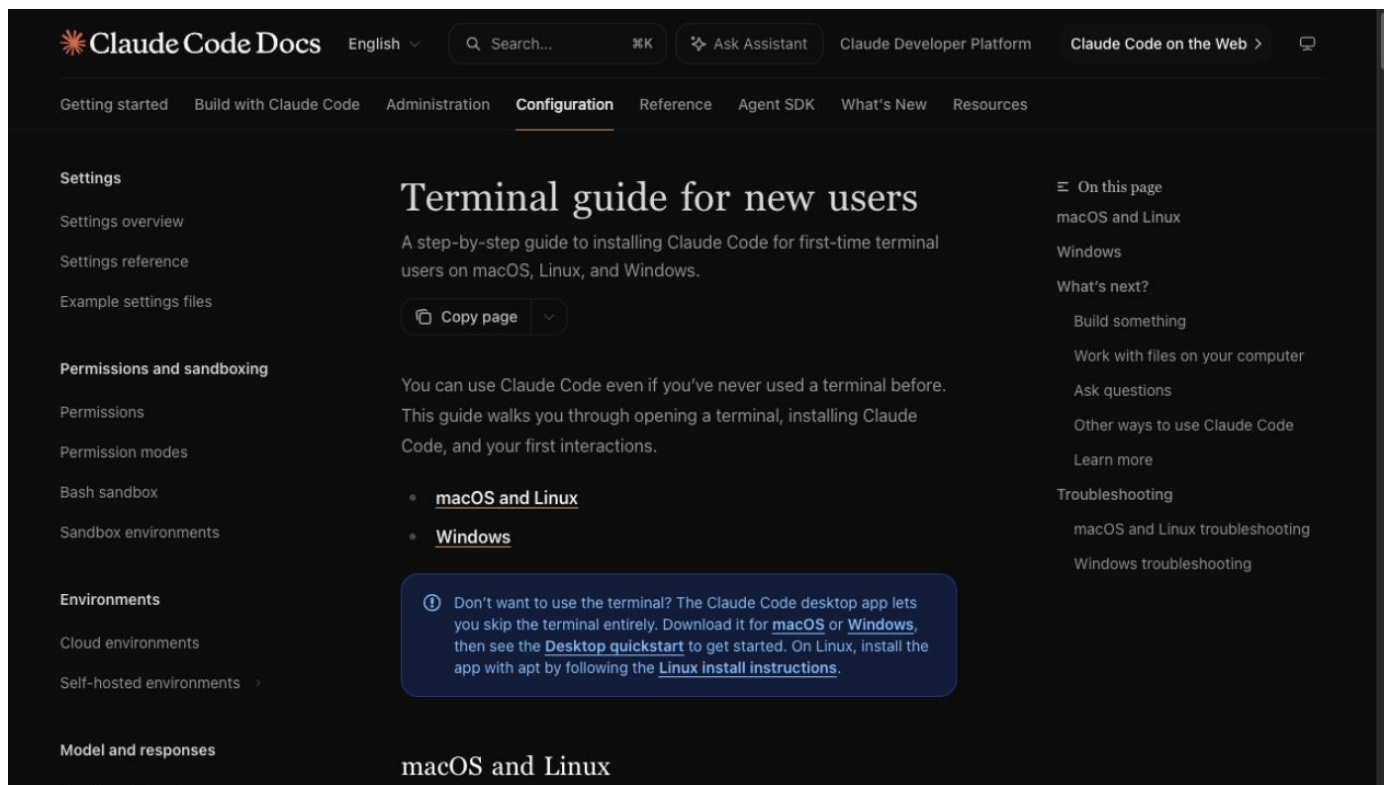
- 허용 파일 한점
- 테스트 명령명 명시
- 생성 결과 diff
- 강사 demo 후 직접 재현



Claude Code · 동일 repo에서 검토 루프

이 화면에서 꼭 볼 것

- 작업 폴더 확인
- 변경 전 계획
- 변경 후 test
- 두 Agent 결과도 사람이 비교



Claude Code 공식 터미널 가이드 화면

규칙 기반 planner

코드 전체보다 경계 한 줄을 먼저 봅니다

- 결정론적 plan
- 파일명만 추출
- unknown도 정상 fixture
- Day2 model로 교체

```
def rule_based_planner(message):  
    match = re.search(r"([\w./-]+\.(txt|md))", message)  
    if match:  
        return {"name": "read_public_text",  
                "arguments": {"path": match.group(1)}}  
    return {"name": "unknown_tool", "arguments": {}}
```

SafeToolExecutor의 경계

코드 전체보다 경계 한 줄을 먼저 봅니다

- 검증 먼저
- registry 함수
- 예외 정규화
- 반환 제약

```
def execute(self, name, args):
    try:
        validate(name, args)
        fn = TOOL_REGISTRY[name]
        data = fn(**args)
        return ToolResult(True, name, data=data)
    except ToolValidationError as exc:
        return ToolResult(False, name,
                          error_code="VALIDATION_ERROR")
```

파일 확장자 allowlist

코드 전체보다 경계 한 줄을 먼저 봅니다

- 목적에 필요한 형식만
- binary 제외
- 정책 메시지
- POLICY_BLOCKED로 매핑!

```
if candidate.suffix.lower() not in {".txt", ".md"}:  
    raise ToolValidationError(  
        "Day 1에서는 .txt와 .md만 허용합니다."  
    )
```

중복 호출 cache

코드 전체보다 경계 한 줄을 먼저 봅니다

- 실행 전 조회
- 성공·실패 정책 결정
- cached 표시
- 외부 쓰기에는 영속 저장

```
call_id = make_call_id(name, args)
if call_id in self._cache:
    prev = self._cache[call_id]
    return replace(prev, cached=True)
result = call_tool()
self._cache[call_id] = result
```

Agent event 한 건

코드 전체보다 경계 한 줄을 먼저 봅니다

- 입력
- 결정
- 결과
- 사람 검토 여부

```
event = {  
    "input": user_message,  
    "planned_call": call,  
    "tool_result": result.to_dict(),  
    "needs_human_review": not result.ok  
}
```

실습 1 · Notebook Run All

12 MIN

강사 화면을 보며
각자 진행합니다

막히면 채팅창에
첫 오류 한 줄만
남겨주세요

- 1 올바른 kernel을 선택한다.
- 2 Restart Kernel을 실행한다.
- 3 Run All을 실행한다.
- 4 첫 실패 셀에서 멈추고 원인을 기록한다.

완료 증거 | 모든 셀 성공

실습 2 · 정상 도구 호출

08 MIN

강사 화면을 보며
각자 진행합니다

막히면 채팅창에
첫 오류 한 줄만
남겨주세요

- 1 sample txt 경로를 입력한다.
- 2 planned_call을 확인한다.
- 3 ToolResult ok와 data를 확인한다.
- 4 원문 글자 수와 경로를 검증한다.

완료 증거 | 정상 event 1건

실습 3 · 실패 4종 재현

10 MIN

강사 화면을 보며
각자 진행합니다

막히면 채팅창에
첫 오류 한 줄만
남겨주세요

1 unknown tool을 실행한다.

2 필수 인자를 지운다.

3 ../ 경로를 넣는다.

4 같은 호출을 두 번 실행한다.

완료 증거 | error_code 3개 + cached 1개

실습 4 · pytest 10개 확인

08 MIN

강사 화면을 보며
각자 진행합니다

막히면 채팅창에
첫 오류 한 줄만
남겨주세요

- 1 terminal을 연다.
- 2 python3 -m pytest -q를 실행한다.
- 3 실패 시 첫 오류만 읽는다.
- 4 10 passed 화면을 저장한다.

완료 증거 | 10 passed

실패 1 · Notebook과 module이 다른 Python

현업에서 자주 깨지는 방식

- import error
- 설치했는데 못 찾는
- terminal은 성공
- kernel은 실패

제가 먼저 고치는 순서

- sys.executable 비교
- interpreter 재선택
- kernel restart
- 같은 .venv

실패 2 · 첫 실패 뒤 계속 실행

현업에서 자주 깨지는 방식

- 연쇄 오류
- 원인 혼동
- output 오염
- 시간 낭비

제가 먼저 고치는 순서

- 첫 traceback
- 가장 위 원인
- 최소 셀 재실행
- Restart & Run All

실패 3 · 테스트를 통과시키려고 정책 완화

현업에서 자주 깨지는 방식

- allowlist 제거
- assert 삭제
- broad except
- expected 변경

제가 먼저 고치는 순서

- 요구사항 재확인
- 최소 수정
- 새 regression test
- 정책 유지

재직자 · Safe loop를 현업 tool에 매핑

교육 도구

- txt 읽기
- marker 세기
- 결과 JSON
- 로컬 cache

현업 예시

- FAQ 조회
- 티켓 분류
- 검토 draft
- request dedupe

Gate 5 · 14:00 점심 전 실행 증거

지금부터 점심시간입니다. 14:55까지 돌아오면 15:00에 Local LLM 연극일부터 바로 시작합니다.

1

Notebook Run All 결과가 남았는가?

2

pytest 10개 통과 화면이 남았는가?

3

내 commit diff에서 바뀐 경계를 한 문장으로 말할 수 있는가?

무료·로컬 LLM과 Adapter

15:00-15:08 강의 · 15:08-15:18 연결 시연 · 15:18-15:35 개인 실습 · 15:35-15:40 확인

지금 배우는 내용	이 차시에서 이해할 것	함께 챙길 내용
Ollama · LM Studio · Fixture Provider	모델이 없어도 같은 구조로 실습을 계속한다	성공 또는 예상된 실패 결과

프로그램이 달라도 generate 계약은 하나로 둡니다

특정 모델과 프로그램이 사라져도 이 실습의 핵심 계약은 그대로 실행되는가?

선택

PC 사양과 UI 선호로 provider 결정

연결

health와 base URL을 확인

변환

adapter가 요청·응답 차이를 흡수

검증

JSON parse와 schema를 순서대로 확인

대체

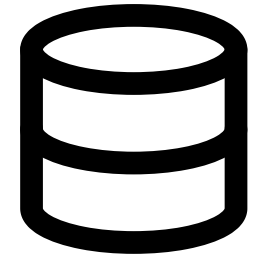
연결 실패 시 fixture로 계속 진행

모델을 교체하기 위한 네 가지 층

용어	쉽게 말하면	오늘 쓰는 곳
Local LLM Model on My Computer	외부 API가 아니라 내 PC에서 실행되는 언어 모델이다.	Ollama에서 qwen3 계열 작은 모델을 실행한다.
Provider Model Access Method	OpenAI·Anthropic·Ollama처럼 모델을 호출하는 서비스 또는 실행 방식이다.	local provider와 fixture provider를 같은 인터페이스로 부릅니다.
Adapter Common Interface	서로 다른 Provider 호출 방식을 하나의 공통 함수로 감싸는 계층이다.	generate(prompt) 함수 뒤에 Ollama·LM Studio·fixture를 숨깁니다.
Fallback Alternative Path	주 경로가 실패했을 때 학습이나 업무를 계속할 대체 경로이다.	Local LLM이 없으면 고정 JSON fixture로 Schema·Graph 설습을 계속한다.

필수 목표는 “모델 설치 성공”이 아니다

provider가 없어도 mock-fixture로 adapter 계약과 예외 처리를 완성한다.



- mandatory: contract
- optional: local model
- no score penalty

무료 실습의 진짜 조건 | 무료는 비용 0원이 아니라 선택권이 남아 있는 구조다.

Ollama는 CLI와 localhost API 경로

모델 pull·serve·generate를 간단한 로컬 흐름으로 제공한다.

- ollama pull
- localhost:11434
- API key 없음

LM Studio는 GUI와 OpenAI-compatible server

모델 관리와 local server를 화면으로 다루기 쉬운 대안이다.

- download model
- start server
- base URL

Jan은 desktop local API 대안

OpenAI-compatible endpoint를 통해 기존 client를 재사용할 수 있다.

- local server
- model selection
- offline-oriented

무료 실습의 진짜 조건 | adapter가 없으면 모델 교체가 곧 프로젝트 재개발이 된다.

Provider adapter가 차이를 감춘다

URL·인증·payload·response를 provider 내부로 모으고 앱에는 공통 result를 준다.

- generate(prompt)
- success data
- normalized error

OpenAI-compatible은 완전 동일을 뜻하지 않는다

model name·지원 필드·tool calling·structured output 차이를 확인한다.

- endpoint
- payload subset
- model capability

작은 모델은 출력 계약을 더 엄격히

JSON parse·schema validate·repair·human review를 별도 단계로 둔다.

- extract JSON
- validate
- bounded repair
- stop

무료 실습의 진짜 조건 | local LLM은 정답이 아니라 데이터·비용·지연시간 사이의 선택지다.

로컬 provider 선택

01

CHECK

RAM·권한

02

CHOOSE

Ollama·GUI

03

VERIFY

health·model

04

FALLBACK

fixture

Ollama 최소 실행

01

INSTALL

공식 배포

02

PULL

작은 모델

03

LIST

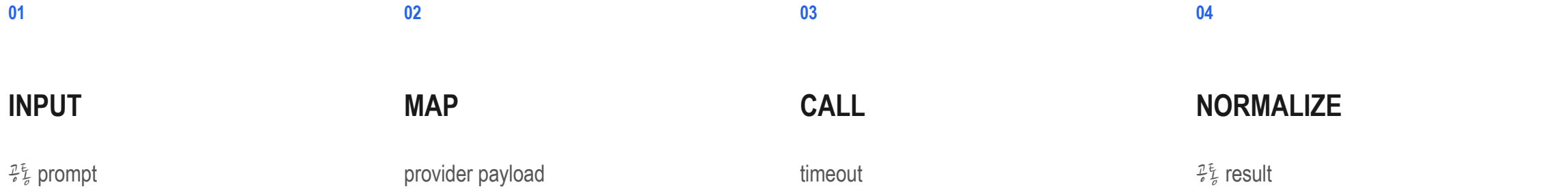
model 확인

04

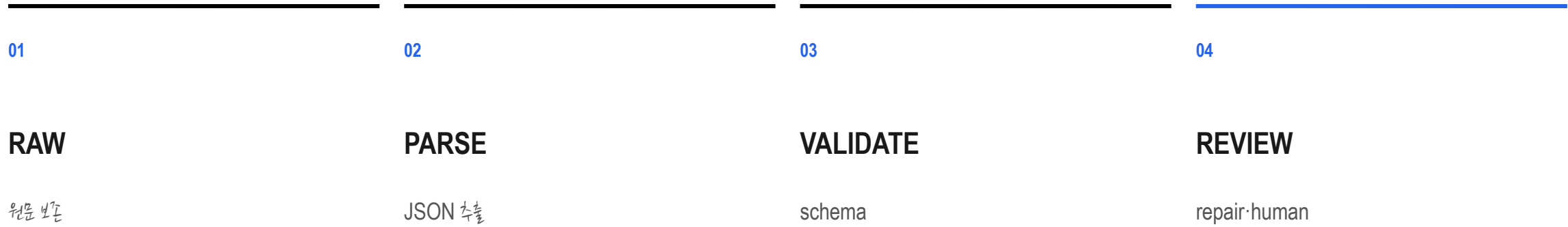
GENERATE

API 호출

Adapter 네 단계



Structured output 처리



모델 오류 분류

01

CONNECT

서버 응답

02

MODEL

tag 응답

03

FORMAT

JSON 미적용

04

QUALITY

근거 누락

세 프로그램의 차이는 화면에서, 공통점은 adapter에서 봅니다

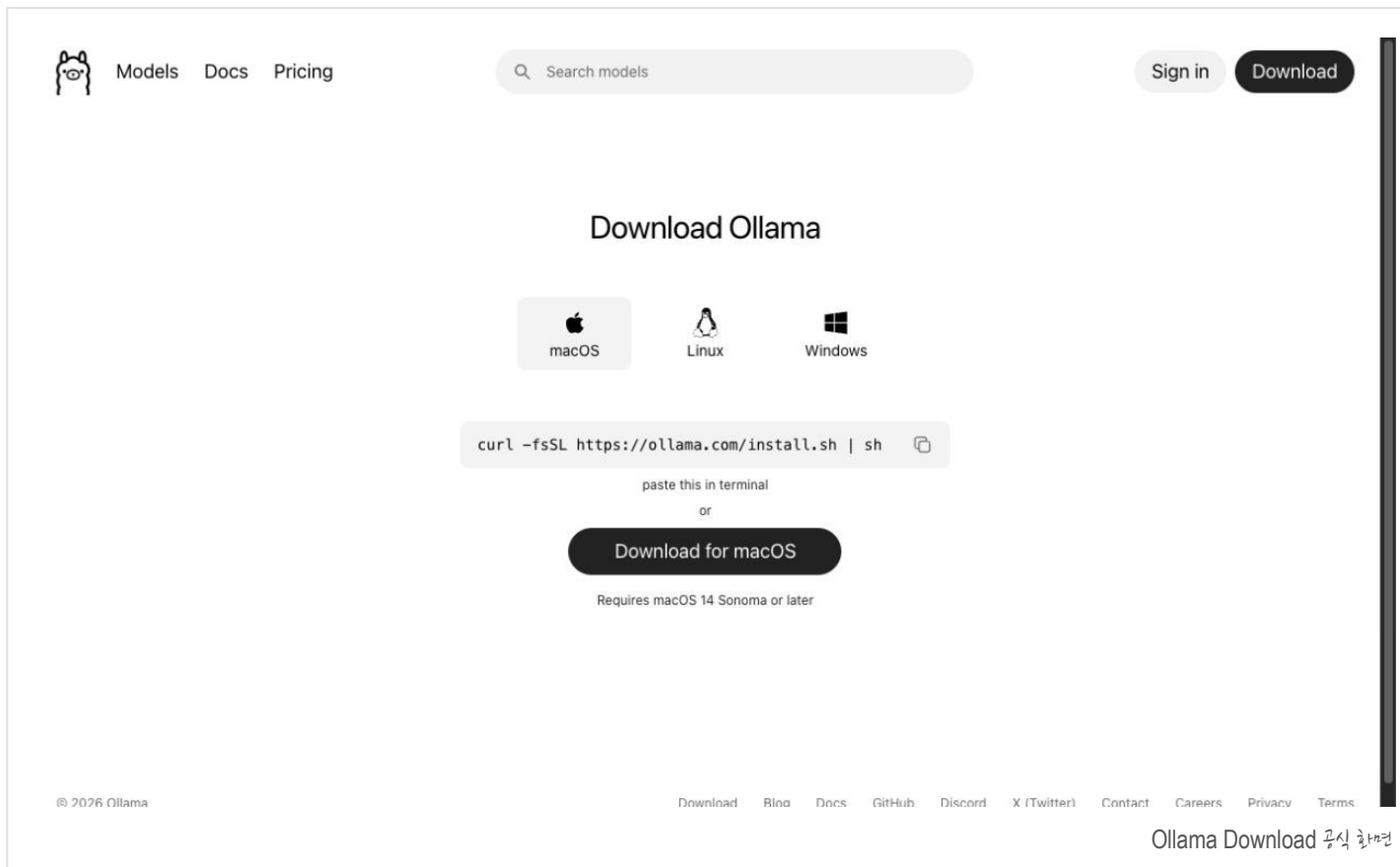
Ollama → model tag → LM Studio server → Jan API server

도구·서비스	도메인·역할	이번 강의에서 쓰는 이유	유사 도구와의 차이
Ollama	CLI·localhost API	명령 몇 줄로 local model과 API를 함께 실행	GUI보다 단순하고 자동화에 유리
LM Studio	GUI·Local Server	모델 탐색과 server 상태를 화면으로 확인	초심자 친화적이나 GUI 조작이 필요
Jan	Desktop Local API	OpenAI-compatible 대체 경로를 하나 더 확보	생성체계에 문서 범위는 도구별 확인
Fixture Provider	설치 없는 고정 응답	모델 설치가 실패해도 Schema·Graph 실행 지속	생성 품질은 평가할 수 없는 복귀 경로

Ollama 다운로드 · OS별 설치

이 화면에서 꼭 볼 것

- 공식 설치 파일
- OS 지원 확인
- 회사 PC 권한
- 수업 전 미리 다운로드



Qwen3 model library · tag 확인

이 화면에서 꼭 볼 것

- 모델 tag를 그대로
- 크기·RAM 고려
- 한 모델만
- 강의 직전 tag 재확인

Models Docs Pricing

Search models

Sign in Download

qwen3

35M Downloads Updated 10 months ago

Qwen3 is the latest generation of large language models in Qwen series, offering a comprehensive suite of dense and mixture-of-experts (MoE) models.

tools thinking 0.6b 1.7b 4b 8b 14b 30b 32b 235b

CLI cURL Python JavaScript

```
ollama run qwen3
```

Applications

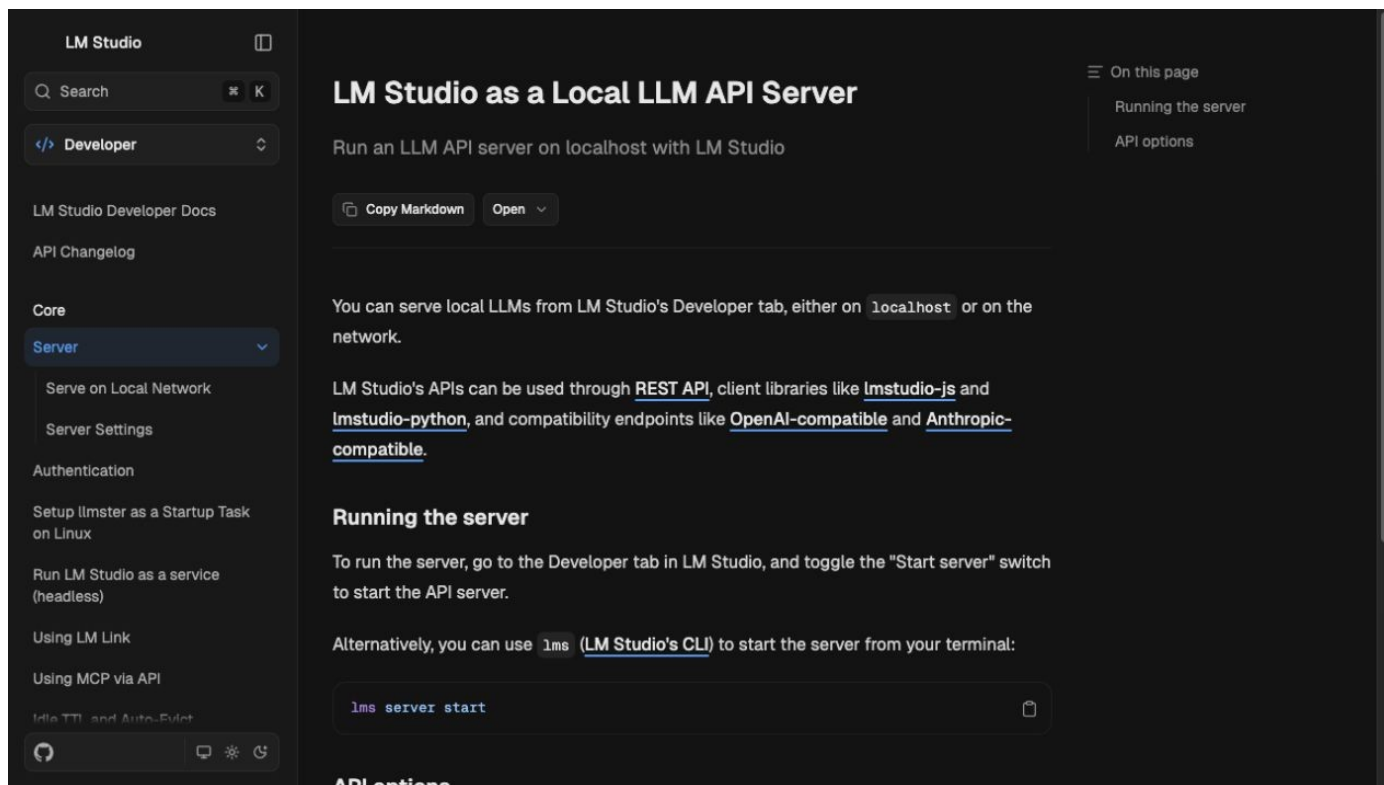
	Claude Code ollama launch claude --model qwen3	
	OpenCode ollama launch opencode --model qwen3	
	Hermes Agent ollama launch hermes --model qwen3	

Ollama Qwen3 model library

LM Studio · Local LLM API server

이 화면에서 꼭 볼 것

- GUI 모델 관리
- server start
- localhost endpoint
- OpenAI-compatible 확인

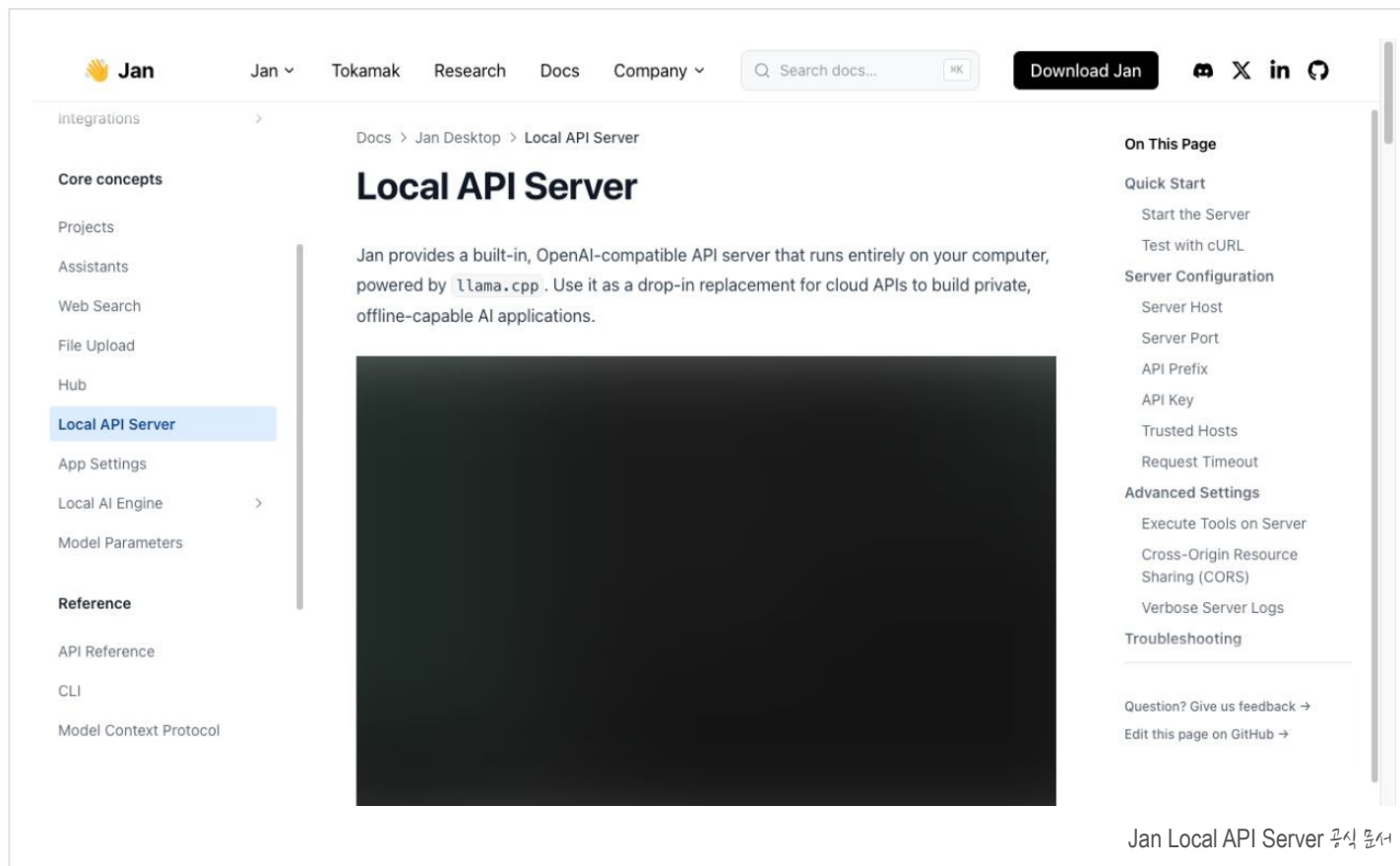


LM Studio local server 공식 문서

Jan · Local API Server

이 화면에서 꼭 볼 것

- desktop 대안
- server 상태
- model selection
- 공통 adapter 연결



Ollama generate payload

코드 전체보다 경계 한 줄을 먼저 봅니다

- localhost
- model tag
- stream 단순화
- timeout 필수

```
payload = {  
    "model": "qwen3:4b",  
    "prompt": prompt,  
    "stream": False  
}  
POST http://localhost:11434/api/generate
```

연결 실패도 result

코드 전체보다 경계 한 줄을 먼저 봅니다

- exception 밖으로 누출 금지
- 안정된 코드
- 상세 원인
- fallback 선택

```
try:
    data = post_json(url, payload, timeout=60)
    return {"ok": True, "data": data}
except URLLError as exc:
    return {"ok": False,
            "error_code": "OLLAMA_UNAVAILABLE",
            "detail": str(exc)}
```

공통 Provider protocol

코드 전체보다 경계 한 줄을 먼저 봅니다

- 앱과 provider 분리
- 같은 입력
- 같은 result
- test double

```
class Provider(Protocol):
    def generate(self, prompt: str) -> ModelResult: ...

providers = {
    "fixture": FixtureProvider(),
    "ollama": OllamaProvider()
}
```

JSON parse와 schema validate

코드 전체보다 경계 한 줄을 먼저 봅니다

- raw 보전
- parse 단계
- typed validate
- 근거 정책

```
raw = provider.generate(prompt)
parsed = extract_json(raw.text)
result = MeetingResult.model_validate(parsed)
if missing_evidence(result):
    return NEEDS_REVIEW
```

bounded repair · 한 번만

코드 전체보다 경계 한 줄을 먼저 봅니다

- 무한 repair 금지
- 오류 정보 제공
- 최대 횟수
- 사람 검토

```
for attempt in range(2):
    try:
        return validate(parse(text))
    except SchemaError as err:
        if attempt == 1: break
        text = provider.generate(repair_prompt(err, text))
return NEEDS_REVIEW
```

실습 1 · Ollama health 확인

08 MIN

강사 화면을 보며
각자 진행합니다

막히면 채팅창에
첫 오류 한 줄만
남겨주세요

- 1 ollama list를 실행한다.
- 2 서버 연결 여부를 확인한다.
- 3 모델이 없으면 설치를 강행하지 않는다.
- 4 fixture lane을 선택해 기록한다.

완료 증거 | provider 상태 1줄

실습 2 · 선택 provider 호출

12 MIN

강사 화면을 보며
각자 진행합니다

막히면 채팅창에
첫 오류 한 줄만
남겨주세요

- 1 prompt를 짧게 입력한다.
- 2 timeout을 지정한다.
- 3 성공이면 response 일부를 확인한다.
- 4 실패면 error_code를 확인한다.

완료 증거 | 성공/실패 result 1건

실습 3 · 공통 contract 비교

08 MIN

강사 화면을 보며
각자 진행합니다

막히면 채팅창에
첫 오류 한 줄만
남겨주세요

- 1 fixture result 필드를 적는다.
- 2 local result 필드를 적는다.
- 3 공통 필드와 provider 전용 필드를 구분한다.
- 4 downstream이 볼 필드를 정한다.

완료 증거 | provider 비교표

실습 4 · 깨진 JSON 복구

10 MIN

강사 화면을 보며
각자 진행합니다

막히면 채팅창에
첫 오류 한 줄만
남겨주세요

- 1 마지막 괄호가 없는 fixture를 넣는다.
- 2 parse 오류를 확인한다.
- 3 repair를 한 번 실행한다.
- 4 두 번째 실패 시 NEEDS_REVIEW로 멈춘다.

완료 증거 | repair 1회 + stop

실패 1 · 큰 모델 다운로드를 기다림

현업에서 자주 깨지는 방식

- 수업 전체 대기
- 디스크 부족
- RAM swap
- 학습 목표 상실

제가 먼저 고치는 순서

- 작은 모델 하나
- 사전 설치
- fixture 즉시 전환
- 모델 비교는 심화

실패 2 · 연결 실패를 traceback으로

현업에서 자주 깨지는 방식

- 노트북 중단
- 학생 원인 파악 어려움
- graph state 유실
- fallback 불가

제가 먼저 고치는 순서

- error_code
- 사용자 메시지
- trace detail
- same result contract

실패 3 · JSON처럼 보이면 통과

현업에서 자주 깨지는 방식

- 필드 누락
- 타입 오류
- 근거 없음
- 날짜 형식 혼합

제가 먼저 고치는 순서

- typed schema
- evidence policy
- date validator
- human review

재직자 · 로컬 LLM 도입 질문

기술 질문

- RAM·속도
- model capability
- API 호환
- 배포 형태

운영 질문

- 데이터 경계
- 업데이트 책임
- 관측·감사
- fallback·SLA

Gate 6 · Local LLM이 꺼져도 계속하기

상황: Ollama 연결이 거부되고, 고실 네트워크에서는 새 모델도 받을 수 없습니다.

1 같은 Provider contract로 어떤 fallback을 연결할까?

2 연결 실패를 traceback 대신 어떤 result로 돌려줄까?

3 fixture로도 끝까지 검증할 출력 계약은 무엇일까?

15:40-16:20

1일차 7차시

LangGraph와 사람 승인

15:40-15:48 강의 · 15:48-16:00 Graph 시연 · 16:00-16:15 개인 실습 · 16:15-16:20 확인

지금 배우는 내용	이 차시에서 이해할 것	함께 쟁길 내용
State · Node · Checkpoint · 승인·수정·거절	중요한 행동 전에는 멈추고 사람이 결정한다	승인·수정·거절 상태

사람 검토를 버튼이 아니라 State 전이로 만듭니다

사람이 개입한 뒤에도 같은 요청과 같은 상태에서 안전하게 이어갈 수 있는가?

상태

다음 node에 필요한 필드만 선언

분기

edge에 retry·review·failed 조건을 둬

종단

interrupt payload로 근거와 초안을 전달

결정

approve·edit·reject event를 저장

재개

같은 thread에서 idempotent하게 이어감

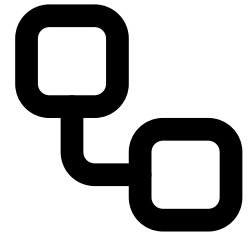
종단과 재개를 설명하는 네 단어

용어	쉽게 말하면	오늘 쓰는 곳
State Workflow Memory	Graph의 각 단계가 읽고 갱신하는 구조화된 현재 상태이다.	transcript·draft·errors·approval·status를 필드로 둡니다.
Node / Edge Step / Route	Node는 한 책임의 작업이고 Edge는 다음 단계로 가는 조건이다.	validate Node가 READY면 review, 오류면 retry 또는 failed로 이동한다.
Checkpoint Saved State	Workflow의 중간 State를 저장해 나중에 같은 지점에서 이어갈 수 있게 한다.	thread_id로 저장된 State를 불러와 승인 뒤 재개한다.
HITL Human in the Loop	중요한 결정이나 외부 변경 전에 사람이 확인하고 승인·수정·거절하는 단계이다.	낮은 confidence Action Item은 reviewer가 근거를 보고 승인한다.

State는 다음 행동의 근거

대화 전체가 아니라 node가 판단에 필요한 최소 구조를 저장한다.

- request_id·step
- draft·errors
- retry·approval·status



Graph가 필요한 이유 | State에 모든 것을 넣는 순간 Graph는 기억장치가 아니라 쓰레기통이 된다.

Node는 한 가지 책임의 함수

입력 state를 읽고 부분 update를 반환하도록 작게 나눈다.

- transcribe
- summarize
- validate
- review

Edge는 다음 node를 정하는 정책

성공·오류·위험·승인 결과에 따라 명시적으로 분기한다.

- conditional route
- END
- retry loop

Checkpoint가 중단과 재개를 가능하게

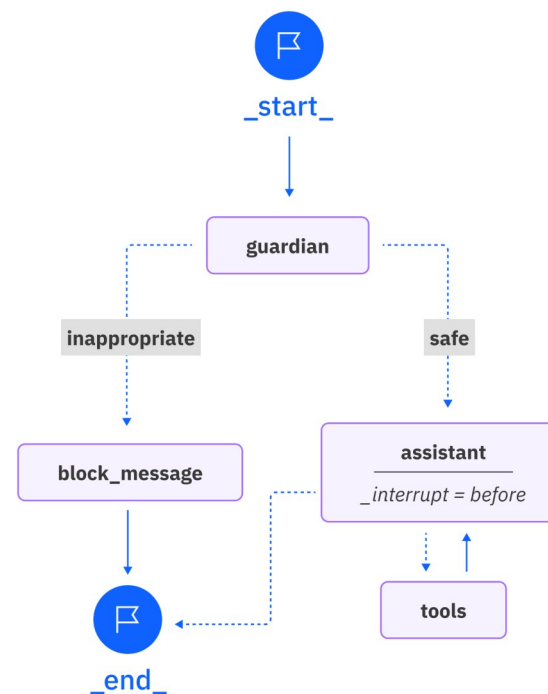
각 step의 state snapshot을 thread 단위로 저장한다.

- checkpoint
- thread_id
- state history

Graph가 필요한 이유 | Human Approval은 버튼이 아니라 근거·결정·책임을 남기는 event다.

Interrupt는 “잠깐 멈춤”이 아니라 업무 상태다

- 어디에서 멈추는가
- 사람이 무엇을 보는가
- 같은 상태로 어떻게 재개하는가



Approve·Edit·Reject는 다른 event

승인은 원안 진행, 수정은 새 payload, 거절은 이유와 종료·재계획을 남긴다.

- decision
- reviewer
- reason
- edited payload

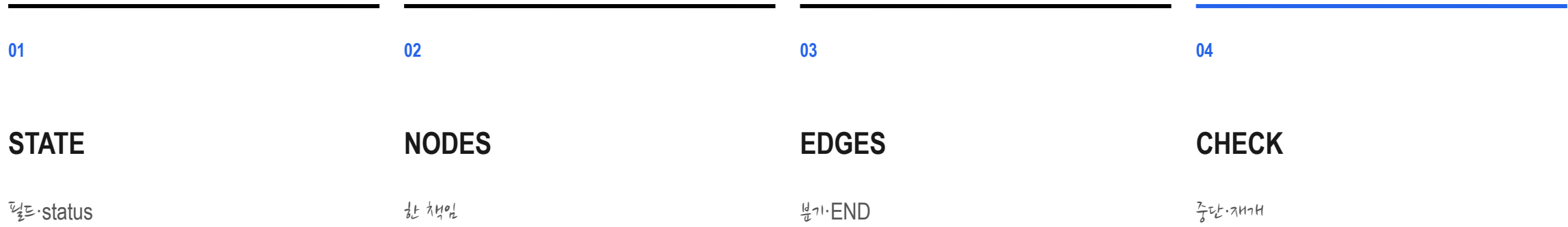
Retry는 일시 오류에만

오류 class·max attempts·backoff·stop을 state에 남긴다.

- transient
- attempts
- next delay
- exhausted

Graph가 필요한 이유 | Idempotency가 없는 resume은 자동화가 아니라 중복 사고의 예약이다.

Graph 설계 순서



오류 routing 순서

01

CLASSIFY

오류 class

02

COUNT

attempt

03

ROUTE

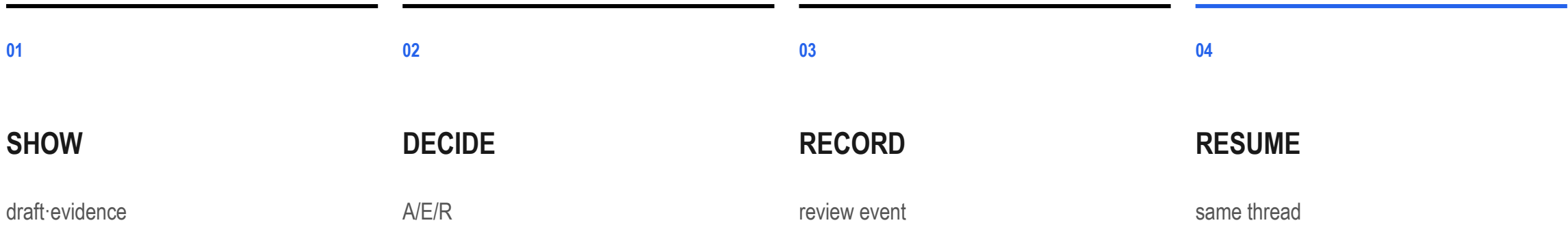
retry·review

04

STOP

terminal status

Human review node



Interrupt payload 설계

01	02	03	04
SUMMARY	TARGET	RISK	OPTIONS
무엇을 하려나	어디에 쓰나	왜 검토하나	승인·수정·거절

Idempotent publish

01

KEY

request_id

02

LOOKUP

기존 성공

03

WRITE

한 번 수행

04

RECORD

external id

LangChain의 조합과 LangGraph의 중단 지점을 구분해서 봅니다

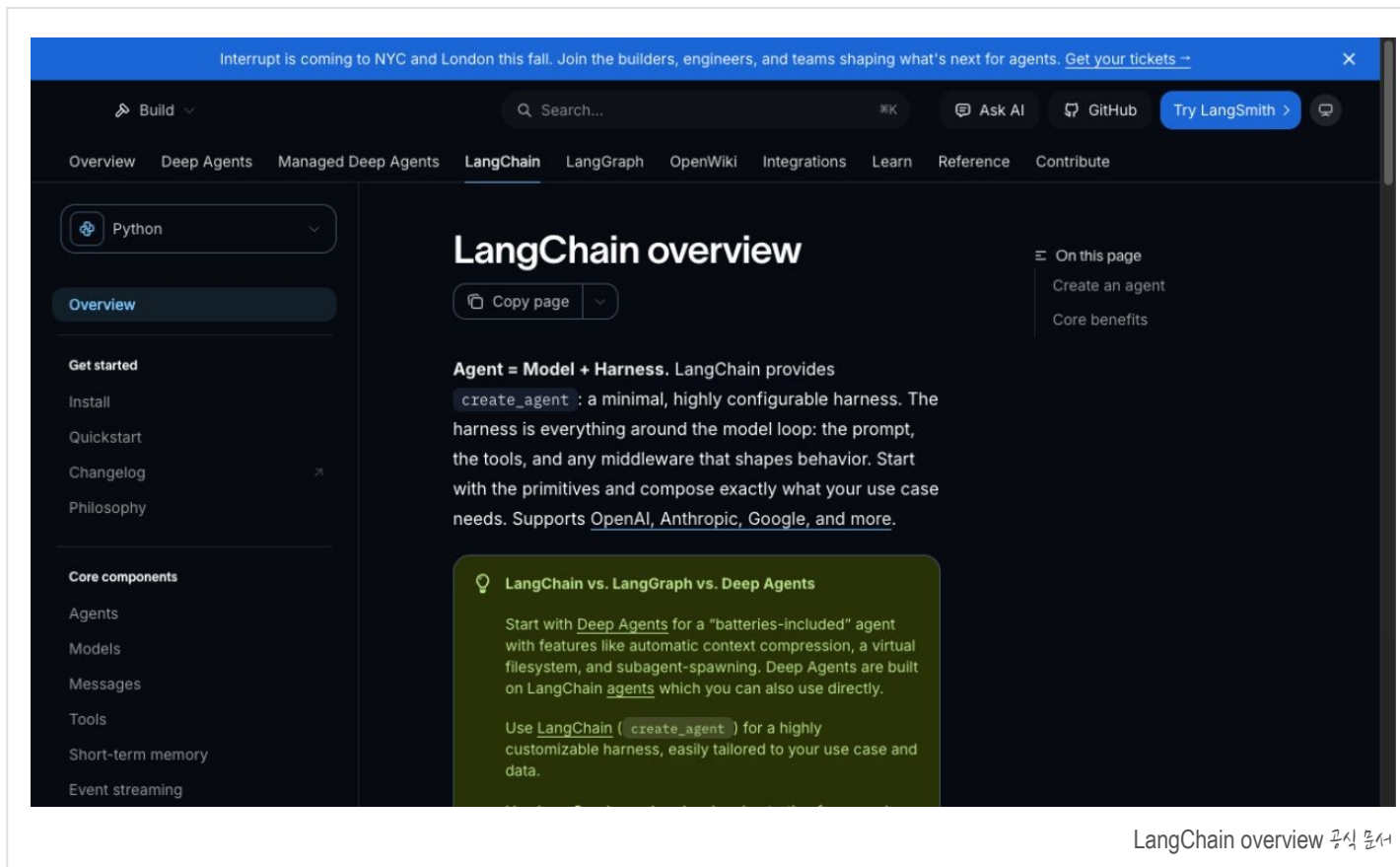
조합 계층 → stateful orchestration → interrupt → persistence

도구·서비스	도메인·역할	이번 강의에서 쓰는 이유	유사 도구와의 차이
LangChain	모델·Prompt·Tool 조합	개별 AI 구성요소를 공통 인터페이스로 연결	단순 호출은 SDK만으로도 가능
LangGraph	상태 기반 Workflow	분기·중단·승인·재개를 State로 명시	짧은 직선 흐름에는 과할 수 있음
LangSmith	관측·평가	trace·dataset·experiment를 같은 실행 계보로 확인	Local JSONL은 무도지만 UI·협업이 약함
Pydantic·pytest	게임식·결정론적 검사	Schema와 실패 조건을 빠르고 재현 가능하게 검증	의미 품질 평가는 별도 rubric 필요

LangChain overview · 조합 계층

이 화면에서 꼭 볼 것

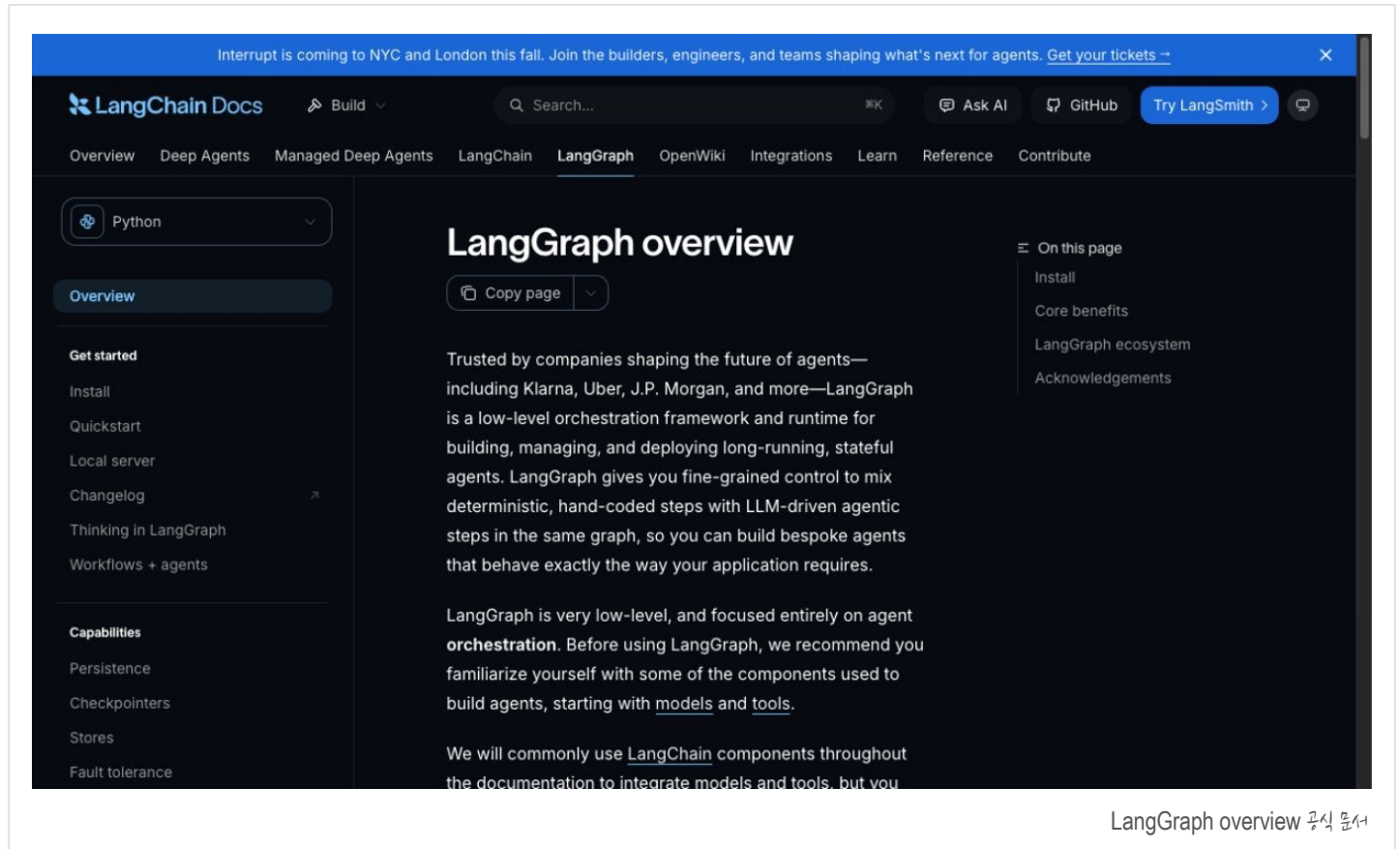
- model·messages·tools
- structured output
- 빠른 조합
- 제어 흐름은 LangGraph



LangGraph overview · stateful orchestration

이 화면에서 꼭 볼 것

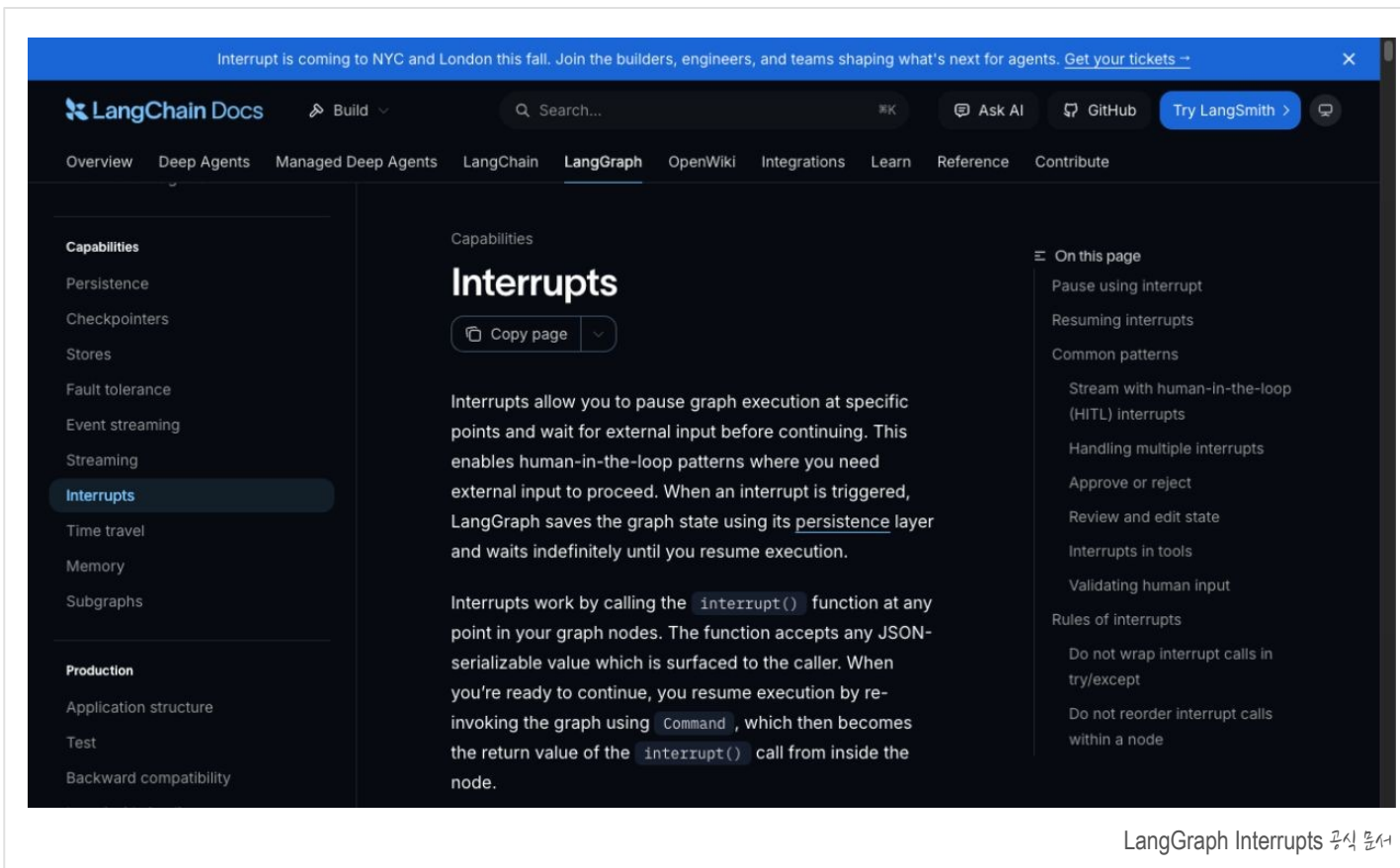
- state·nodes·edges
- durable execution
- human-in-the-loop
- 장기 실행 workflow



LangGraph interrupts · 사람 입력에서 중단

이 화면에서 꼭 볼 것

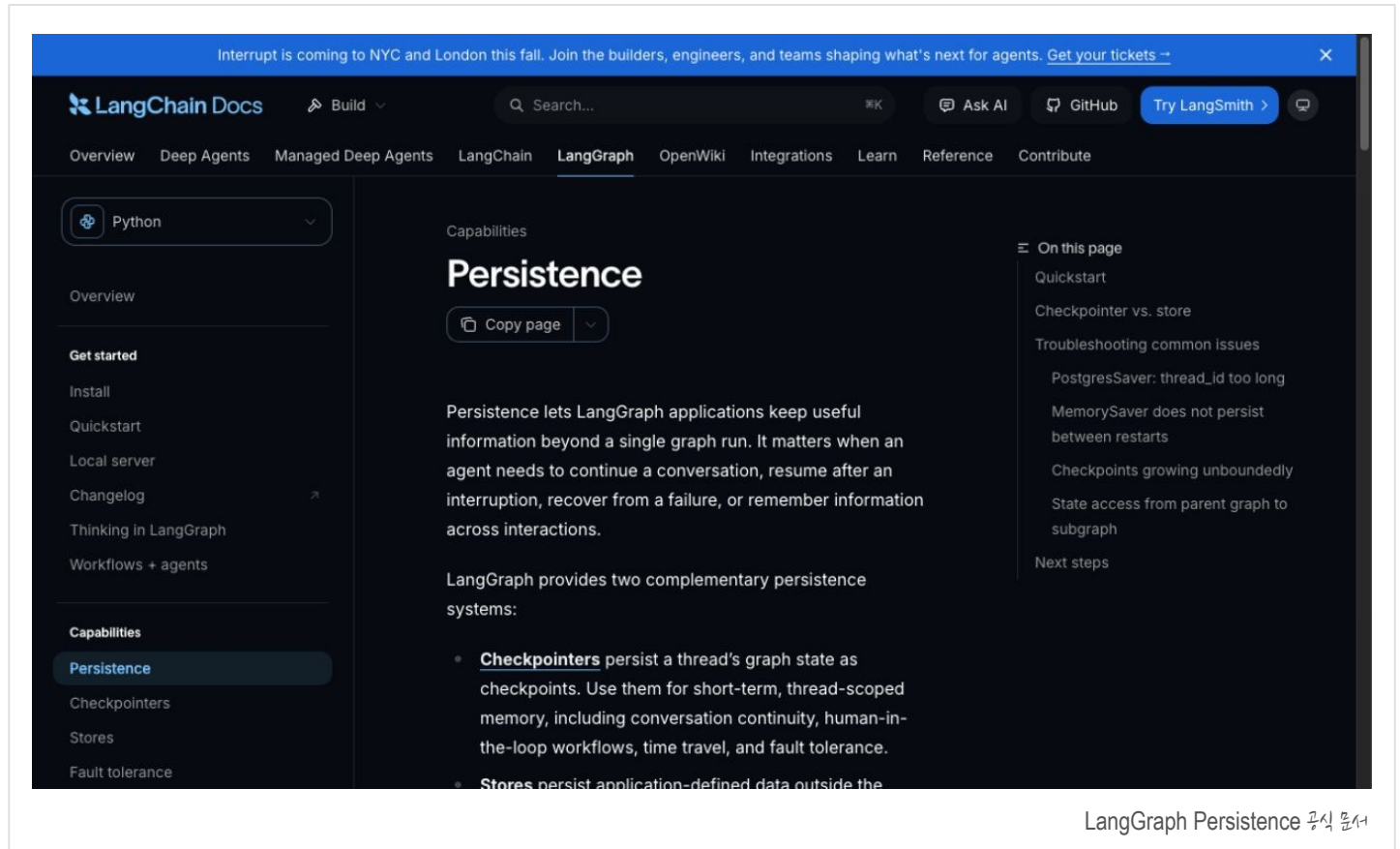
- interrupt payload
- Command resume
- JSON-serializable
- side effect idempotent



LangGraph persistence · checkpoint와 thread

이 화면에서 꼭 볼 것

- checkpointer
- thread_id
- state history
- fault tolerance



State schema

코드 전체보다 경계 한 줄을 먼저 봅니다

- 다음 node에 필요한 필드
- raw data 유지
- 오류 누적
- terminal status

```
class State(TypedDict):  
    request_id: str  
    transcript: list[dict]  
    draft: dict | None  
    errors: list[dict]  
    retry_count: int  
    approval: dict | None  
    status: str
```

오류 route 함수

코드 전체보다 경계 한 줄을 먼저 봅니다

- status 기반
- bounded retry
- 정책 오류 제외
- 명시적 failed

```
def route_after_validate(state):  
    if state["status"] == "READY":  
        return "human_review"  
    if state["status"] == "TRANSIENT_ERROR" and  
state["retry_count"] < 2:  
        return "retry"  
    return "failed"
```

Interrupt로 검토 요청

코드 전체보다 경계 한 줄을 먼저 봅니다

- 검토 정보 한 화면
- 직렬화 가능한 값
- 버튼만 전달 금지
- 결정은 event로

```
review = interrupt({
  "draft": state["draft"],
  "evidence": state["evidence"],
  "target": state["publish_target"],
  "options": ["approve", "edit", "reject"]
})
```

Command로 같은 thread 재개

코드 전체보다 경계 한 줄을 먼저 봅니다

- thread_id 유지
- checkpoint load
- resume value
- new thread와 구분

```
config = {"configurable": {"thread_id": request_id}}
first = graph.invoke(input_state, config)
# interrupt에서 멈춤
second = graph.invoke(Command(resume={"decision": "approve"}),
config)
```

승인 event

코드 전체보다 경계 한 줄을 먼저 봅니다

- 누가
- 무슨 결정
- 이유
- 수정 payload·시각

```
approval = {  
    "decision": "edit",  
    "reviewer": "role-reviewer",  
    "reason": "담당자 근거 부족",  
    "edited_payload": corrected,  
    "at": iso_time  
}
```

실습 1 · State 필드 설계

08 MIN

강사 화면을 보며
각자 진행합니다

막히면 채팅창에
첫 오류 한 줄만
남겨주세요

- 1 입력·중간·오류·승인 필드를 나눈다.
- 2 각 node가 읽을 필드를 표시한다.
- 3 terminal status를 세 개 정한다.
- 4 불필요한 chat history를 제거한다.

완료 증거 | State schema 1개

실습 2 · Node와 edge 카드

10 MIN

강사 화면을 보며
각자 진행합니다

막히면 채팅창에
첫 오류 한 줄만
남겨주세요

- 1 transcribe부터 publish까지 카드를 놓는다.
- 2 각 node 한 책임을 쓴다.
- 3 성공 edge를 연결한다.
- 4 오류·review edge를 다른 색으로 연결한다.

완료 증거 | graph sketch

실습 3 · Retry 정책표

08 MIN

강사 화면을 보며
각자 진행합니다

막히면 채팅창에
첫 오류 한 줄만
남겨주세요

- 1 오류 네 종류를 적는다.
- 2 각 오류의 retry 여부를 정한다.
- 3 max attempts와 backoff를 적는다.
- 4 exhausted 후 상태를 정한다.

완료 증거 | retry matrix

실습 4 · Reviewer 역할극

12 MIN

강사 화면을 보며
각자 진행합니다

막히면 채팅창에
첫 오류 한 줄만
남겨주세요

- 1 A는 Agent, B는 Reviewer가 된다.
- 2 초안과 evidence를 비교한다.
- 3 approve/edit/reject와 이유를 남긴다.
- 4 역할을 바꾸고 낮은 confidence 사례를 반복한다.

완료 증거 | review event 2건

실패 1 · State에 모든 것을 저장

현업에서 자주 깨지는 방식

- 거대한 message history
- binary audio 포함
- 개인정보 장기 보존
- node coupling

제가 먼저 고치는 순서

- 최소 structured state
- 원본은 보호 저장소
- reference ID
- node별 계약

실패 2 · interrupt를 try/except로 감쌌

현업에서 자주 깨지는 방식

- 중단 신호를 오류 처리
- 재개 불가
- state 혼란
- 반복 호출

제가 먼저 고치는 순서

- 공식 interrupt 규칙
- 직렬화 payload
- 같은 thread
- 명시적 resume

실패 3 · 승인 전에 외부 쓰기

현업에서 자주 깨지는 방식

- 재실행 중복
- 오류 확산
- 사람이 취소 불가
- 감사 추적 부재

제가 먼저 고치는 순서

- draft·preview
- interrupt
- approved payload
- idempotent publish

재직자 · Human Approval 운영표

승인 기준

- 개인정보 flag
- 낮은 confidence
- 외부 발신
- 금악·계악 영향

운영 증거

- reviewer role
- SLA
- decision reason
- audit log

Gate 7 · 승인 뒤 같은 요청이 다시 왔습니다

상황: reviewer가 승인한 직후 네트워크가 끊겼고, 같은 request_id로 publish가 다시 호출됐습니다.

1

어떤 State와 checkpoint에서 재개 할까?

2

승인 event에서 어떤 근거와 결정을 읽을까?

3

새로 발행할까, 저장된 성공 결과를 돌려줄까?

STT와 LangSmith

16:20-16:28 강의 · 16:28-16:38 Trace 시연 · 16:38-16:53 개인 실습 · 16:53-17:00 판단

지금 배우는 내용	이 차시에서 이해할 것	함께 챙길 내용
STT 품질 · Trace · Dataset · 평가	실행 기록으로 READY와 HOLD를 결정한다	STT 품질 기준 + READY/HOLD 결정

관측은 READY와 HOLD를 가를 때 끝납니다

다음 버전을 배포하지 말아야 할 근거를 trace와 dataset에서 바로 찾을 수 있는가?

입력

Managed 또는 Local STT의 segment를 받음

품질

무음·반복·이름·날짜 오류를 flag

계보

node·LLM·tool을 trace tree로 연결

비교

같은 dataset으로 baseline과 candidate 실행

결정

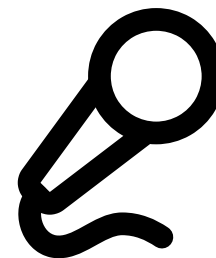
품질·안전·지연 기준으로 READY/HOLD

관측과 배포 판단을 잇는 네 단어

용어	쉽게 말하면	오늘 쓰는 곳
STT Speech to Text	사람의 음성을 문자와 타임스탬프로 변환하는 기술이다.	faster-whisper가 audio를 segment 배열로 변환한다.
Trace Execution Record	한 요청 안에서 Node·LLM·Tool이 어떤 입력과 출력을 냈는지 연결한 실행 기록이다.	LangSmith run tree에서 Graph 아래 child run을 확인한다.
Dataset / Experiment Fixed Inputs / Compared Runs	Dataset은 평가 입력 묶음이고 Experiment는 같은 입력으로 특정 버전을 실행한 결과 묶음이다.	day2-v1 데이터셋에 baseline과 candidate를 각각 실행한다.
Release Gate Ship or Hold Rule	품질·안전·지연시간 기준을 모두 통과할 때만 배포를 허용하는 규칙이다.	Schema 98%·근거 연결 90%·오발행 0건을 모두 만족해야 READY이다.

Managed 회의 기능은 현업의 현재 출발점

Google Meet 등은 전사·노트 기능을 제공하지만 계정·edition·관리자 설정에 의존한다.



- 빠른 활용
- 정책·보존
- 지원 언어·edition 확인

배포 전에 보는 신호 | STT가 들리면 LLM은 들린 입력을 더 그럴듯하게 정리한다.

수업의 local STT는 원리를 이해하는 최소 실습

faster-whisper 또는 whisper.cpp로 audio→segment contract를 직접 확인한다.

- offline 가능
- timestamp
- model·device 선택

STT 품질은 WER 하나로 끝나지 않는다

업무에서는 이름·숫자·날짜·Action Item 근거 오류를 별도로 본다.

- empty-repetition
- entity error
- timestamp
- human correction

배포 전에 보는 신호 | Trace는 과거 기록이 아니라 다음 실험 질문을 만드는 재료다.

LangSmith trace는 실행의 계보

graph run **아래** node·LLM·tool의 input/output·latency·error를 연결한다.

- parent-child
- metadata
- tags
- error

민감 데이터는 trace 전에 줄인다

교육 계정에는 공개·합성 샘플만 전송하고 원문·secret을 masking 한다.

- anonymize
- sampling
- local JSONL
- retention 확인

Dataset이 평가 입력을 고정

golden example과 실패 사례를 dataset으로 만들고 experiment를 반복한다.

- input
- reference output
- metadata
- version

배포 전에 보는 신호 | 평가는 점수를 내는 일이 아니라 READY와 HOLD를 가르는 운영 결정이다.

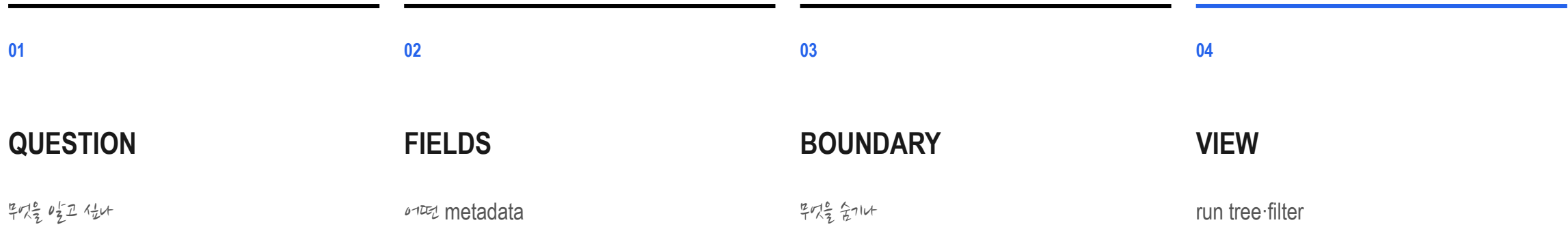
전체 pipeline



STT 실행 순서



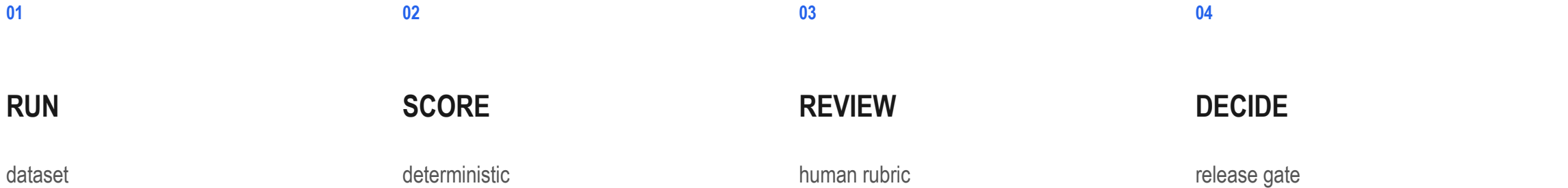
Trace 설계 순서



Dataset 구축

01	02	03	04
SEED	FAIL	VERSION	EXPERIMENT
golden 3-5건	예외 추가	변경 기록	같은 입력 비교

평가 루프



STT와 LangSmith 화면은 하나의 품질 루프로 연결됩니다

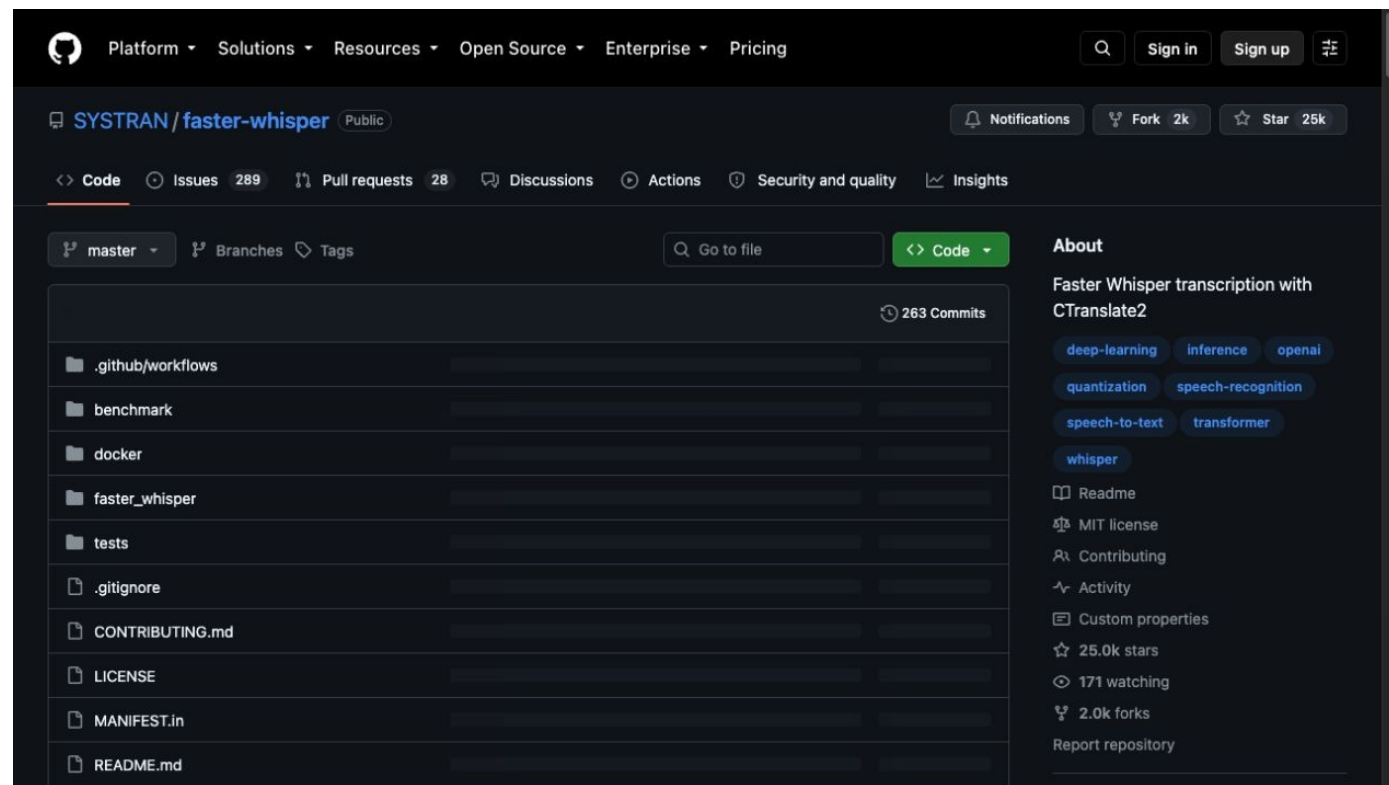
Local STT 두 경로 → Managed 노트 → trace tree → experiment

도구·서비스	도메인·역할	이번 강의에서 쓰는 이유	유사 도구와의 차이
Google Meet	Managed 회의 STT	이미 제공되는 전사·노트를 현업 출발점으로 사용	Zoom·Teams도 유사하지만 구독 정책이 다름
faster-whisper	Python Local STT	segment·timestamp를 코드에서 직접 다루기 쉬움	whisper.cpp보다 Python 통합이 편함
whisper.cpp	경량 CLI STT	낮은 사양·다양한 환경의 local 대안	Python pipeline 연결은 추가 작업
LangSmith Trace	실행 관측	node·LLM·tool의 지연·오류 계보를 한 화면에서 확인	민감 데이터는 masking 또는 local trace
LangSmith Eval	Dataset·Experiment	같은 입력으로 baseline과 candidate를 비교	pytest는 구조 검사에 강하고 의미 평가는 제한

faster-whisper · Python 기반 local STT

이 화면에서 꼭 볼 것

- Whisper 재구현
- model·device·compute_type
- segments generator
- 수업 전 model 준비

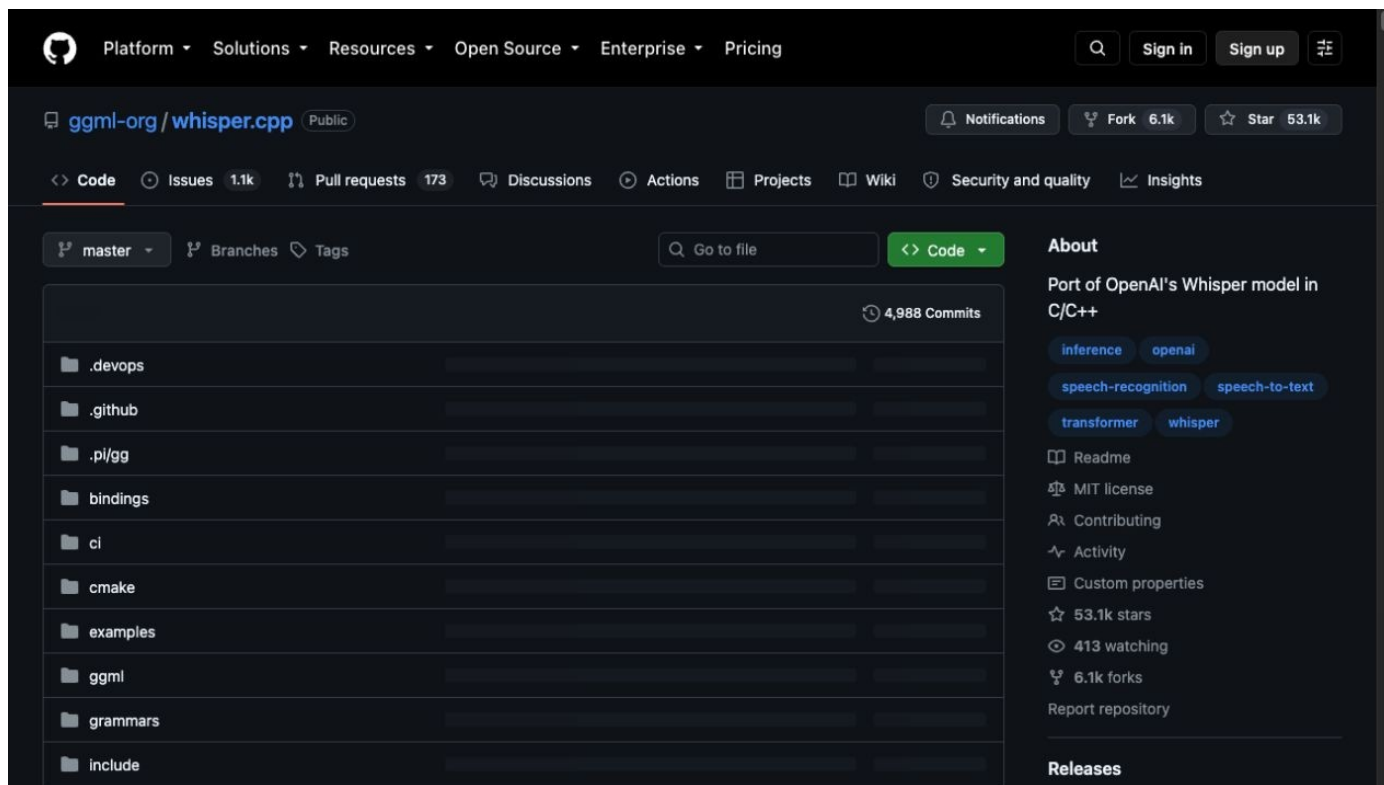


faster-whisper 공식 GitHub

whisper.cpp · 경량 local 대안

이 화면에서 꼭 볼 것

- C/C++ 기반
- 다양한 환경
- CLI 실습 대안
- 모델 파일 사전 준비

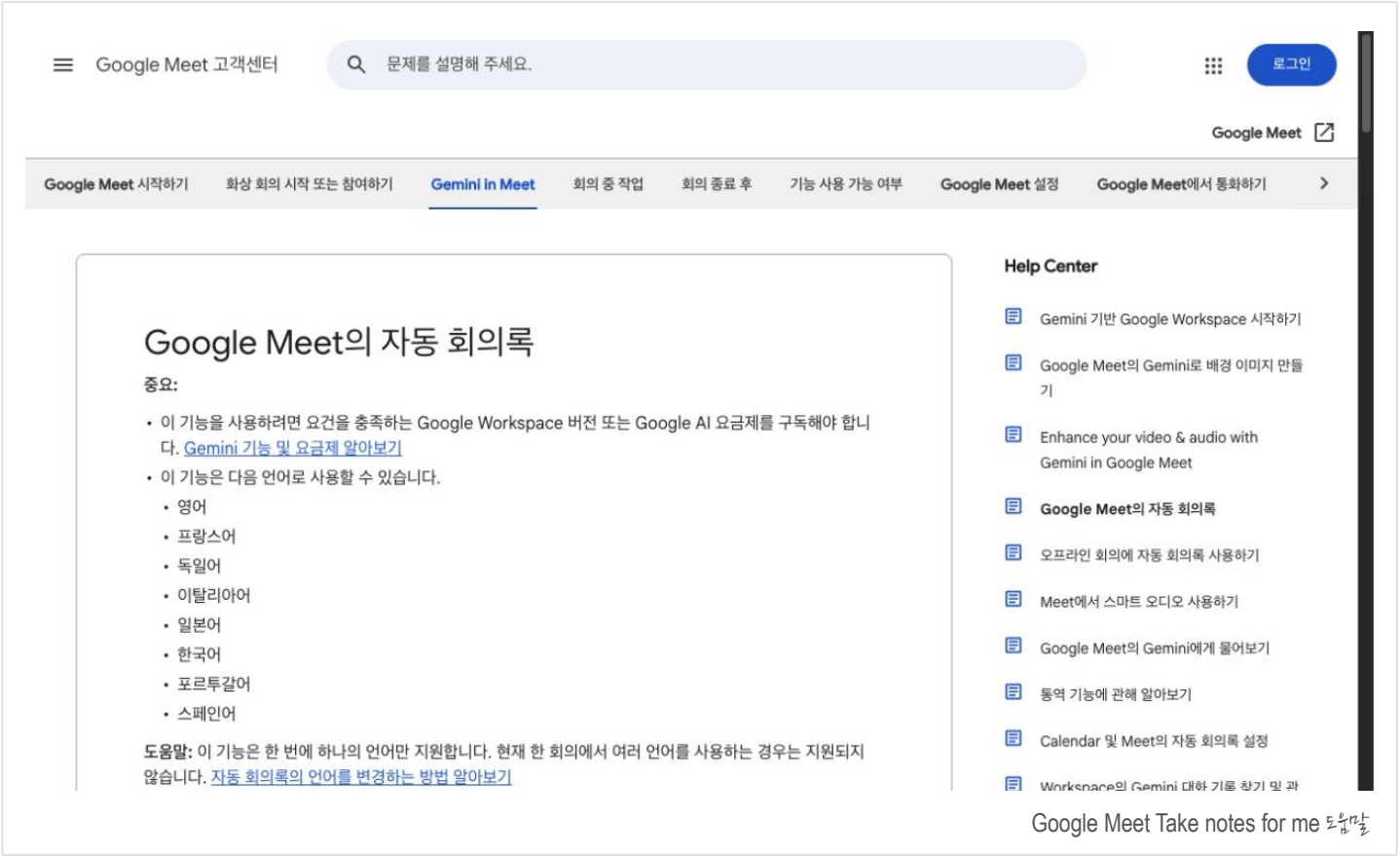


whisper.cpp 공식 GitHub

Google Meet 자동 노트 · 만들기보다 통합하기

이 화면에서 꼭 볼 것

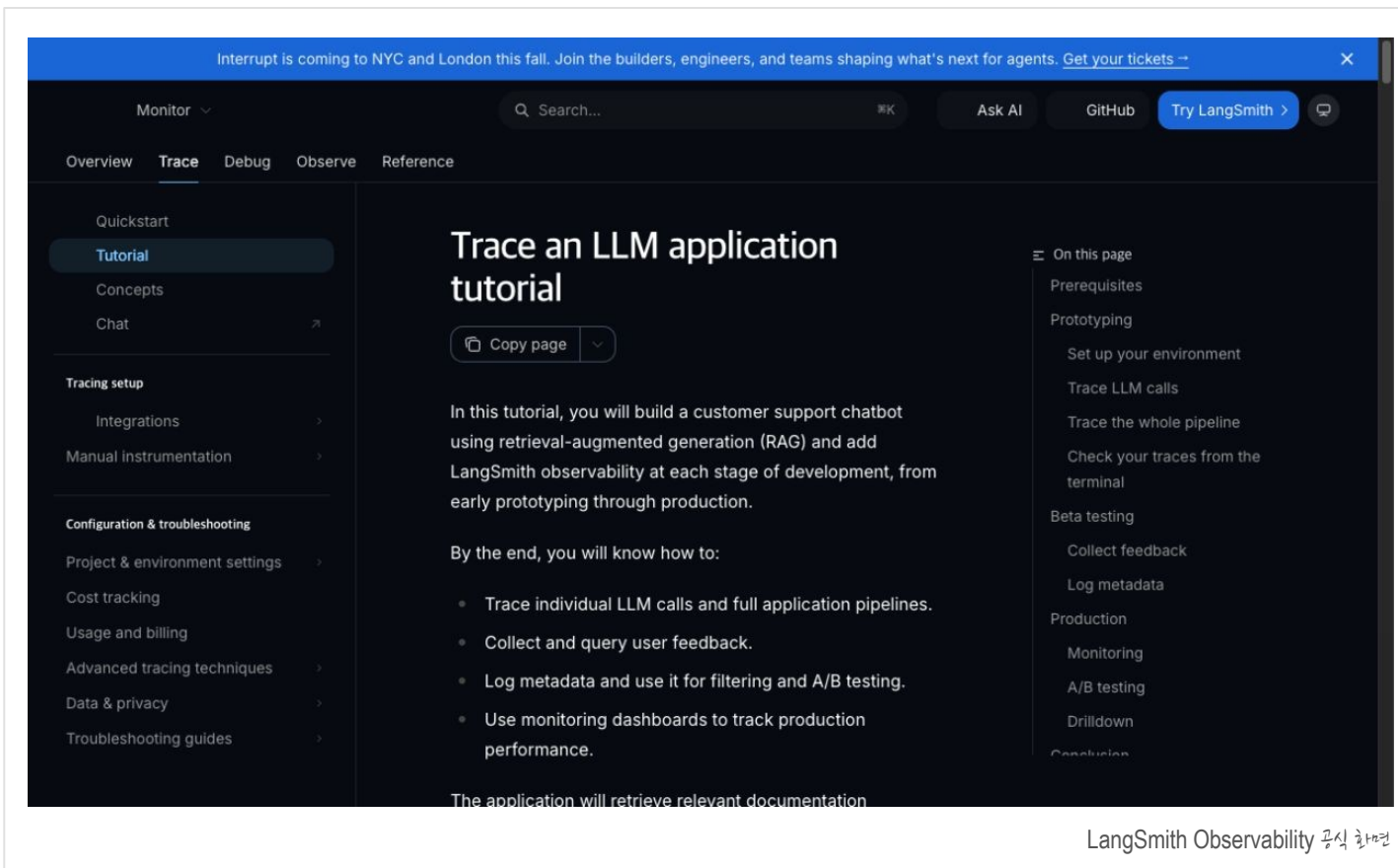
- 구독·관리자 설정 의문
- 기존 전사·요약 활용
- local은 예외·동제 학습
- 지원 범위 강의 전 확인



LangSmith observability · trace tree

이 화면에서 꼭 볼 것

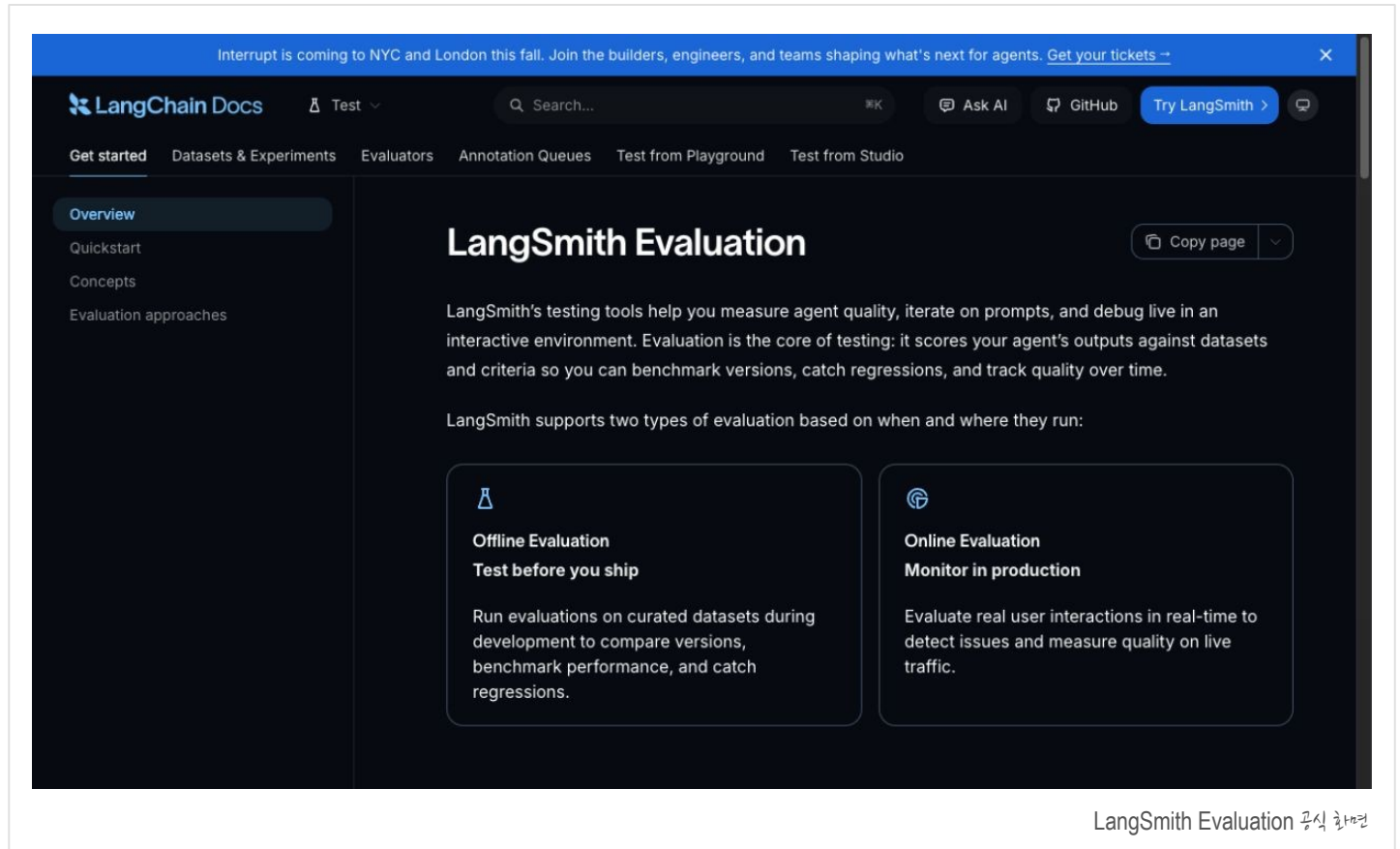
- node별 input/output
- latency·token·error
- metadata·tag
- 공개 샘플만



LangSmith evaluation · dataset과 experiment

이 화면에서 꼭 볼 것

- offline evaluation
- online monitoring
- human feedback
- baseline 비교



faster-whisper 최소 호출

코드 전체보다 경계 한 줄을 먼저 봅니다

- CPU int8 경로
- language 명시
- segment timestamp
- generator 소버

```
from faster_whisper import WhisperModel
model = WhisperModel("small", device="cpu", compute_type="int8")
segments, info = model.transcribe(audio_path, language="ko")
rows = [{"start": s.start, "end": s.end, "text": s.text}
        for s in segments]
```

STT 품질 flag

코드 전체보다 경계 한 줄을 먼저 봅니다

- 무음
- 반복
- 지나치게 희소
- 사람 검토

```
flags = []
if not text.strip(): flags.append("EMPTY")
if repeated_ratio(text) > 0.35: flags.append("REPETITION")
if duration > 0 and len(text) / duration < 0.5:
    flags.append("T00_SPARSE")
if flags: status = "NEEDS_REVIEW"
```

LangSmith 환경 변수 경계

코드 전체보다 경계 한 줄을 먼저 봅니다

- 프로젝트 분리
- key 환경 변수
- 교육 샘플만
- tracing off fallback

```
LANGSMITH_TRACING=true  
LANGSMITH_PROJECT=ipa-k-work-copilot  
# LANGSMITH_API_KEY is set outside notebook  
# never print or commit the key
```

Trace metadata

코드 전체보다 경계 한 줄을 먼저 봅니다

- dataset version
- provider·model
- thread 연결
- 민감성·fallback

```
metadata = {  
    "dataset_version": "day2-v1",  
    "provider": "ollama",  
    "model": "qwen3:4b",  
    "thread_id": request_id,  
    "contains_pii": False,  
    "fallback_used": False  
}
```

결정론적 evaluator

코드 전체보다 경계 한 줄을 먼저 봅니다

- 근거 연결률
- schema pass
- 계산 재현
- release threshold

```
def evidence_rate(output):
    claims = output["decisions"] + output["action_items"]
    linked = [c for c in claims if c.get("evidence_ids")]
    return len(linked) / max(1, len(claims))

def schema_pass(output):
    return MeetingResult.model_validate(output) is not None
```

실습 1 · Managed vs Local 선택표

07 MIN

강사 화면을 보며
각자 진행합니다

막히면 채팅창에
첫 오류 한 줄만
남겨주세요

- 1 현재 조직의 회의 도구를 적는다.
- 2 전사·요약 제공 여부를 적는다.
- 3 계정·보존·권한 제약을 적는다.
- 4 local 최소 실습이 필요한 이유를 쓴다.

완료 증거 | Managed/Local 비교표

실습 2 · STT quality policy

08 MIN

강사 화면을 보며
각자 진행합니다

막히면 채팅창에
첫 오류 한 줄만
남겨주세요

- 1 무음 기준을 정한다.
- 2 반복 기준을 정한다.
- 3 이름·날짜 오류를 정한다.
- 4 사람 검토로 보내는 조건을 적는다.

완료 증거 | STT review rule

실습 3 · Trace에서 답할 질문

08 MIN

강사 화면을 보며
각자 진행합니다

막히면 채팅창에
첫 오류 한 줄만
남겨주세요

1 가장 느린 node를 묻는다.

2 가장 자주 실패한 오류를 묻는다.

3 사람이 많이 수정한 필드를 묻는다.

4 각 질문에 필요한 metadata를 적는다.

완료 증거 | 질문 3개 + 필드

실습 4 · Evaluation dataset 설계

10 MIN

강사 화면을 보며
각자 진행합니다

막히면 채팅창에
칭 오류 한 줄만
남겨주세요

- 1 정상 회의 2건을 정한다.
- 2 무음·모호한 발자·근거 누락 사례를 추가한다.
- 3 reference JSON을 정한다.
- 4 schema·evidence evaluator를 연결한다.

완료 증거 | dataset 5건 설계

실패 1 · STT 결과를 정답으로 간주

현업에서 자주 깨지는 방식

- 이름·숫자 오류
- 무음 반복
- 화자 혼동
- timestamp 유실

제가 먼저 고치는 순서

- quality flags
- 원문 audio link
- 사람 correction
- segment evidence

실패 2 · 모든 trace에 원문 전송

현업에서 자주 깨지는 방식

- 개인정보
- 회사 기밀
- 보존 정책 불명
- 교육 계정 혼용

제가 먼저 고치는 순서

- 공개·합성 샘플
- masking
- metadata 최소화
- local JSONL fallback

실패 3 · trace만 보고 평가 완료

현업에서 자주 깨지는 방식

- 관찰은 가능
- 정답 기준 없음
- 회귀 판단 불가
- release 결정 없음

제가 먼저 고치는 순서

- dataset
- evaluator
- baseline
- gate·HOLD

재직자 · 운영 dashboard 질문

매일 볼 것

- failure rate
- review queue
- latency
- cost·fallback

배포 전 볼 것

- dataset score
- schema·evidence
- false publish
- rollback 기준

17:30-18:00 · Q&A와 Exit Ticket



오늘 막힌 부분을 먼저 복구하고, 마지막에는 내 Workflow를 READY 또는 HOLD로 판단합니다.

1

내 흐름에서 자동 실행과 사람 승인의 경계는 어디인가?

2

실패 뒤 다시 시작할 checkpoint와 fallback은 무엇인가?

3

어떤 trace-dataset 신호가 나오면 다음 배포를 HOLD할 것인가?